
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 15:52:42
Файлов исходного кода: 85
Суммарный объём кода: 1.15 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 24. src/data/tickets/ticket1_5.ts
- 25. src/data/tickets/ticket14_20.ts
- 26. src/data/tickets/ticket21_24.ts
- 27. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 28. src/components/ArticulationPointsViz.tsx
- 29. src/components/BellmanFordViz.tsx
- 30. src/components/BfsViz.tsx
- 31. src/components/ChapterImage.tsx
- 32. src/components/ChapterNav.tsx
- 33. src/components/ChapterView.tsx
- 34. src/components/DfsBridgesSimulator.tsx
- 35. src/components/DijkstraViz.tsx
- 36. src/components/DPVisualizer.tsx
- 37. src/components/EulerSimulator.tsx
- 38. src/components/ExpressQuiz.tsx
- 39. src/components/FloydViz.tsx
- 40. src/components/GraphTraversalViz.tsx
- 41. src/components/HeapViz.tsx
- 42. src/components/hints/demoKit.tsx
- 43. src/components/hints/demosExtra.tsx
- 44. src/components/hints/demosGraph.tsx
- 45. src/components/hints/demosStruct.tsx
- 46. src/components/hints/MiniDemo.tsx
- 47. src/components/hints/SimulatorModal.tsx
- 48. src/components/hints/TermPopover.tsx
- 49. src/components/JohnsonViz.tsx
- 50. src/components/KosarajuViz.tsx
- 51. src/components/KruskalSimulator.tsx
- 52. src/components/MnemonicCards.tsx
- 53. src/components/MobileToc.tsx
- 54. src/components/Navbar.tsx
- 55. src/components/PlanarityDemo.tsx
- 56. src/components/PythonCompiler.tsx
- 57. src/components/QueueViz.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/85: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`./github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
```

```
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+   ....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

- ```
# На Ubuntu политика ImageMagick иногда запрещает экспорт
# разрешаем то, что нужно пайплайну (text -> pdf)
for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml.d/*; do
  if [ -f "$policy" ]; then
    sudo sed -i 's/rights="none" pattern="\{PDF,PSD\}" rights="write" /rights="write" /' "$policy"
  fi
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.density:-300}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages.pdf` | \$(du -h --max-depth=1 | sort -n | tail -n 1) |"
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages.png` | \$(du -h --max-depth=1 | sort -n | tail -n 1) |"
 echo "| PNG-страниц | \$(ls tmp/export-pages | wc -l) |"
 } >> "\$GITHUB_STEP_SUMMARY"

```
+++ b/.github/workflows/rebuild-pdf.yml
```

```
@@ -42,15 +42,15 @@ jobs:
```

```
  steps:
```

```
    - name: Checkout
```

```
-     uses: actions/checkout@v4
```

```
+     uses: actions/checkout@v6
```

```
    with:
```

```
      fetch-depth: 0
```

```
      persist-credentials: true
```

```
    - name: Setup Node.js
```

```
-     uses: actions/setup-node@v4
```

```
+     uses: actions/setup-node@v7
```

```
    with:
```

```
-       node-version: "22"
```

```
+       node-version: "24"
```

```
      cache: npm
```

```
    - name: Install ImageMagick + DejaVu fonts
```

```
@@ -97,7 +97,7 @@ jobs:
```

```
      } >> "$GITHUB_STEP_SUMMARY"
```

```
    - name: Upload artifacts (PDF + TXT)
```

```
-     uses: actions/upload-artifact@v4
```

```
+     uses: actions/upload-artifact@v7
```

```
    with:
```

```
      name: full-code-export
```

```
      path: |
```

```
@@ -107,7 +107,7 @@ jobs:
```

```
      retention-days: 30
```

```
    - name: Upload artifacts (PNG-страницы)
```

```
-     uses: actions/upload-artifact@v4
```

```
+     uses: actions/upload-artifact@v7
```

```
    with:
```

```
      name: full-code-pages-png
```

```
      path: tmp/export-pages/*.png
```

`public/favicon.svg`

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
```



---

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
 <div className="min-h-screen bg-slate-950 text-slate
```

```
 <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
 {activeTab === "guide" ? (
```

```
 <>
```

```
 <MobileToc
```

```
 selectedChapterId={activeChapterId}
```

```
 setSelectedChapterId={goTo}
```

```
 currentTitle={activeChapter.title}
```

```
 index={index}
```

```
 total={chapters.length}
```

```
 />
```

```
 <div className="flex flex-col lg:flex-row max
```

```
 {/* Сайдбар только на десктопе – на мобильном
```

```
 <div className="hidden lg:block lg:w-80 shr
```

```
 <Sidebar selectedChapterId={activeChapter
```

```
 </div>
```

```
 <main id="main-content" className="flex-1 m
```

```
 {/* Шапка главы с быстрыми переходами */}
```

```
 <div className="flex items-center justify
```

```
 <div className="min-w-0">
```

```
 <div className="text-[10px] uppercase
```

```
 {activeChapter.category || "Универс
```

```
 </div>
```

```
 <h2 className="text-base sm:text-xl f
```

```
 {activeChapter.title}
```

```
 </h2>
```

```
 </div>
```

```
 <ChapterNav prev={prev} next={next} ind
```

```
 </div>
```

```
 <div className="h-1 w-full bg-slate-800 r
```

```
 <div
```

```
 });
 }

```

### Patch относительно `main`

```
````diff
```

```
diff --git a/src/App.tsx b/src/App.tsx
```

```
index 2091f33..4583756 100644
```

```
--- a/src/App.tsx
```

```
+++ b/src/App.tsx
```

```
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
```

```
import { ChapterNav } from "../components/ChapterNav";
```

```
import { SimulatorModal } from "../components/hints/SimulatorModal";
```

```
import { SimulatorHub } from "../components/SimulatorHub";
```

```
+import { PythonCompiler } from "../components/PythonCompiler";
```

```
export default function App() {
```

```
-   const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
```

```
+   const [activeTab, setActiveTab] = useState<"guide" | "simulator">();
```

```
   const [activeChapterId, setActiveChapterId] = useState<string>();
```

```
   const [modalVizId, setModalVizId] = useState<string>();
```

```
@@ -95,7 +96,7 @@ export default function App() {
```

```
    </main>
```

```
    </div>
```

```
  </>
```

```
-   ) : (
```

```
+   ) : activeTab === "simulator" ? (
```

```
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
```

```
      <SimulatorHub
```

```
        onOpen={setModalVizId}
```

```
@@ -105,6 +106,8 @@ export default function App() {
```

```
    </main>
```

```
  </>
```

```
    </main>
```

```
+   ) : (
```

```
+   <PythonCompiler />
```

```

        k-0">
        <Sparkles className="w-6 h-6" />
    </div>
    <div className="min-w-0 flex-1">
        <h1 className="text-xl font-extrabold text-white">
            Универсальное пособие
        </h1>
        <p className="text-xs text-indigo-400 font-extrabold">
            Подготовка к экзаменам: Алгоритмы, Структуры
        </p>
    </div>
</div>

<div className="flex items-center w-full lg:w-auto">
    >
    <button
        id="tab-guide-btn"
        onClick={() => setActiveTab('guide')}
        className={`flex-1 lg:flex-none flex items-center justify-center text-white font-extrabold cursor-pointer transition-200 whitespace-nowrap ${
            activeTab === 'guide'
                ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
                : 'text-slate-400 hover:text-white'
            }`}
    >
        <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
        id="tab-simulator-btn"
        onClick={() => setActiveTab('simulator')}
        className={`flex-1 lg:flex-none flex items-center justify-center text-white font-extrabold cursor-pointer transition-200 whitespace-nowrap ${
            activeTab === 'simulator'
                ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
                : 'text-slate-400 hover:text-white'
            }`}
    >
        <Cpu className="w-4 h-4" /> Тренажёры

```

```
        id="download-html-btn"
        onClick={saveBook}
        disabled={busy}
        className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
      >
        {busy ? <Loader2 className="w-4 h-4 animate
      </button>
    </div>
  </div>
</header>
);
};
},
```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
```

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) {
    return error.message;
  }
  return String(error);
}
```

```

        setRuntimeMessage("Не удалось выполнить код – проп
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>

                <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]>
                    <section className="rounded-2xl border border-slate-200 p-4">

```

```

        aria-label="Код Python"
        className="block w-full min-h-[360px] res
        none focus:ring-2 focus:ring-inset focus:r
        placeholder="Напишите Python-код..."
    />

<div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
        Ввод для input() • по одной строке на к
    </label>
    <textarea
        id="python-stdin"
        value={stdin}
        onChange={(event) => setStdin(event.targ
        spellCheck={false}
        rows={2}
        placeholder={'Например:\nАлиса'}
        className="block w-full resize-y bg-tra
        eholder:text-slate-700"
    />
</div>
</section>

<section className="rounded-2xl border border
    <div className="flex items-center justify-b
        <div className="flex items-center gap-2 t
            <Terminal className="w-4 h-4 text-emera
        </div>
        <span className={`inline-flex items-cente
            {runtimeReady && <CheckCircle2 classNam
            {runtimeReady ? "готов" : "не загружен"
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px]
    x] leading-6 text-slate-300">
        {output}
    </pre>
    <div className="border-t border-slate-800 p

```

```
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = null
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
```



```

+     setCode(STARTER_CODE);
+     setStdin("");
+     setOutput("Нажмите «Запустить», чтобы увидеть резу
+ });
+
+ return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+         <div className="flex flex-col sm:flex-row sm:i
+         <div>
+             <div className="inline-flex items-center g
+                 <Terminal className="w-4 h-4" /> Боковая
+             </div>
+             <h2 className="text-2xl sm:text-3xl font-e
+             <p className="text-slate-400 mt-2 max-w-2x
+                 Пишите небольшой код и запускайте его пр
+                 который переводит CPython в WebAssembly.
+             </p>
+         </div>
+         <a
+             href="https://pyodide.org/"
+             target="_blank"
+             rel="noreferrer"
+             className="inline-flex items-center gap-1.5
+         >
+             Как это работает <ExternalLink className="
+         </a>
+     </div>
+
+     <div className="grid xl:grid-cols-[minmax(0,1.
+         <section className="rounded-2xl border borde
+         <div className="flex flex-wrap items-cente
+             <div className="flex items-center gap-2">
+                 <span className="w-2.5 h-2.5 rounded-f
+                 <span className="text-sm font-bold tex
+                 <span className="text-[11px] text-slate
+             </div>
+             <div className="flex items-center gap-2">

```

```

+         <label className="block px-4 pt-3 text-[
+             Ввод для input() • по одной строке на
+         </label>
+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.ta
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border borde
+         <div className="flex items-center justify-l
+             <div className="flex items-center gap-2
+                 <Terminal className="w-4 h-4 text-emerg
+             </div>
+             <span className={`inline-flex items-centr
+                 {runtimeReady && <CheckCircle2 classNa
+                 {runtimeReady ? "готов" : "не загружен
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800
+     </section>
+ </div>
+
+     <div className="mt-5 grid md:grid-cols-3 gap-3
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-ind
+             <span>Ctrl или Cmd + Enter запускает код.

```

```

    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

3. **\*\*Строгая структура контента:\*\***

- Заголовок с цветным бейджем (например, `bg-indigo-100`)
- Определение/правило в центре (`border-blue-500` или `border-blue-500/2px`)
- Обязательная сетка аналогий (2 колонки: неформальный текст и код)
- "Душные" детали (код, формулы, сложные доказательства)

4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

---

ФАЙЛ 4/85: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 610 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/85: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
}
```

```

 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/85: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1000

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5173
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в один файл

| Команда | Шаг | Результат |

## Универсальное пособие • полный исходный код

- на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`)
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

src/

- App.tsx – оболочка, привязка глав к визуализации
- components/ – интерактивные симуляторы (Действия)
- data/ – контент пособия (главы/билеты)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновление)
- .github/workflows/ – автоматизация

...

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика, параграфы `

`.

Универсальное пособие • полный исходный код

>  
> Всего глав в пособии: **\*\*25\*\***.

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

| Билет         | Тема                           | Статус                                 |
|---------------|--------------------------------|----------------------------------------|
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Книжка читается вхолостую.  |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (книжки)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|    |                                          |   |   |   |
|----|------------------------------------------|---|---|---|
| 12 | 12. Графы. Точки сочленения              | 1 | - | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра     | 3 | - | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 |   |   |
| 16 | 16. Графы. Кратчайший путь. Флойд        | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона             | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал      | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима        | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка      | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП             | 4 | - | - |
| 22 | 22. Z-функция                            | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик                 | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач     | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы           | - | - | - |
| -  | Алгоритмы на карте                       | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                       | - | - | - |
| -  | Мосты в графах: код + визуализация       | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-mar`\*\*** — раскрыть или удалить.

ФАЙЛ 9/85: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
```



```
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz)?.scrollIntoView();
 });
 });
 }, []));

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div className={compilerOpen ? "max-lg:pb-[58dvh] max-lg:pt-[58dvh]" : ""}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 />
 </>
) : (
 <MainContent
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />
)}
 </div>
 </div>
);

```

```

 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>
) : (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
)}

 <footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное
 </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

{compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>
);
}

```

```

*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;
 }
 }
}

```

```
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/85: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 К

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
}
```

```

 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {

```

```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```

-----  
}

-----  
ФАЙЛ 16/85: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В  
-----

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

=====

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

=====

-----  
ФАЙЛ 17/85: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМЕР: 10 КБ  
-----

```
import { Chapter } from "../types";
```



```

 <p class="text-slate-300 text-sm mb-4">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```



каждой вставки чинят дерево, чтобы оно <em>  
</li>

<li><span class="text-white font-bold">  
менно быть кривым. Но каждый раз, когда мы  
ровнее становится.</li>

</ul>

</div>

<!-- 2. Аналогии -->

<div class="grid grid-cols-1 xl:grid-cols-2 gap-

<div class="bg-slate-800 p-6 rounded-lg bor-

<div class="absolute -top-3 left-4 bg-s-  
border-emerald-500 shadow-md">

Аналогия 1: стопка бумаг на столе

</div>

<p class="text-slate-300 text-sm">У теб  
скиваешь его и, поработав, кладёшь <span cl  
неделю самые нужные бумаги сами собой оказ  
ет ровно это: «взял → положил наверх».</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg bor-

<div class="absolute -top-3 left-4 bg-r  
der-rose-500 shadow-md">

Аналогия 2: тропинка через газон

</div>

<p class="text-slate-300 text-sm">Ты ид  
ода дорожка сама укорачивалась вдвое: чем ч  
оплачивает все следующие. Пойдёшь один раз

</div>

</div>

<!-- 3. Поворот -->

<div class="bg-slate-800/60 p-6 rounded-xl bord-

<h3 class="text-lg font-bold text-white mb-

<p class="text-slate-300 text-sm mb-4">Повор  
движение, три перевешенные ссылки</span> и

<pre class="bg-slate-950 p-4 rounded-lg ove  
ading-relaxed"> 40

```

 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">деда
 <td class="py-3 pr-3">деда
 <td class="py-3 pr-3">сам х
 <td class="py-3">х стал кор
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">
аво-право)</td>
 <td class="py-3 pr-3">
 <td class="py-3">х поднялся
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bolder">
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3">
 <td class="py-3">р и г стал
 </tr>
 </tbody>
 </table>
 </div>

```

```

 <p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
 <p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>

```

```
</div>
```

```
<!-- 5. Главная ловушка -->
```

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
 <h3 class="text-lg font-bold text-rose-300 m
 <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
 <div class="grid grid-cols-1 lg:grid-cols-2
 <div class="bg-slate-950 rounded-lg p-4
 <p class="text-rose-400 font-bold t

```

```
<pre class="bg-slate-950 p-4 rounded-lg over
0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```

```
 40
 /
 30
 /
20
/
10
```

```
 30
 / \
 10 40
 \ \
 20
```

```
 10
 \ \
 30
 / \
 20 40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-1
 <p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
ах идея такая:</p>
 <div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">H
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">д
 </div>
 </div>
```

```

 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда не нашли
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сбалансировано
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сбалансировано
 rotate(x);
 }
 }
 return x; // теперь x — корень
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
 let cur = root, last = null;

```

```

open: border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Последний
 енный или преемник). Splay делают всегда, да
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Treap дер
 play не хранит вообще ничего и держит балан
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 }
}

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```



```

 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 В некоторых городах существуют односторонние улицы, что может повлиять на направление движения по графу. Например, в графе, представляющем дорожную сеть, односторонние улицы могут быть modeled как directed edges. Это может повлиять на алгоритмы поиска кратчайшего пути, которые должны учитывать направление движения.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-rose-300 italic
 <p class="text-xl font-mono text-white"
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-40
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-indigo-300 italic

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под землей.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
 </div>
</div>
</div>
</div>
</section>`,
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```



```

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
```

```

content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-purple-300 italic
 <p class="text-xl font-mono text-white"
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведенн
аче B, то B – это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

```

 <div class="p-3 bg-slate-950 rounded
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

* <span class="text-green-400 font-
шел-вошёл-вышел).

* <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.</s
ng> ровно раз и вернуться домой.

```

```
<h2 class="text-3xl font-bold text-white">Мосты
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded"
 <span class="text-rose-400 font"
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
```

```
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся – граф p
 </p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg border"
 <div class="absolute -top-3 left-4 bg-slate-900
 border-emerald-500 shadow-md">
```

Мост: Единственная веревка

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу
 Из пещеры и нет других выходов – то
 Если верёвка (ребро v-u) оборвётся -
 Если бы из пещеры и был чёрный ход
</p>
```



```

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

```

```

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Запуск

</div>

</details>

</div>

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
 -white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 – эталон не
 Если внутри графа спрятан K5 – забудь,
 </p>
 </div>

```

```

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6, E=9
 xt-white">9 ≤ 12</code> – проходит, но он все равно не планарен
 Почему? Потому что у двудольного графа K3,3
 Отсюда более строгое ограничение: <code class="text-white">9 > 8</code>
 </p>
</div>

```

```

</div>
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6, E=9
 xt-white">9 ≤ 12</code> – проходит, но он все равно не планарен
 Почему? Потому что у двудольного графа K3,3
 Отсюда более строгое ограничение: <code class="text-white">9 > 8</code>
 </p>
</div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Доказательство непланарности K5 (Скрыть)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400">Доказательство
 <p>
 Пусть K5 планарен. $V = 5, E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Каждая грань ограничена минимумом
 </p>
 </div>

```

еются к точкам сочленения. То есть, **ег** - это тупик со степенью 1).

</p>

</div>

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

</div>

<p class="text-slate-300 text-sm mb-4">

Представь город, где два района соединены перекрестком. Если один из районов будет хрупкостью системы.

</p>

</div>

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной сети. Если один из них выйдет из строя, это повлияет на работу всей сети (мостом). Сами свитчи — это точки сочленения.

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg border border-slate-500">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
```

<div class="p-5 text-sm text-slate-300 space-y-2">

<p class="text-blue-400 font-bold">Особые примечания

<p>

```

 <strong class="text-emerald-400">SCC-зависимые "конгломераты".
 Как точка сочленения рушит Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая сильная компонента и есть критическая SCC! Если удалить эту компоненту, одно слабое звено изолирует целые касты SCC.

</div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/85: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp";

// We want to combine all of them, but overwrite 7, 11, 12, 13
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24,
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-

```

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

```
 "Волна от источника: сначала все соседи, потом со
```

```
 formal:
```

```
 "Очередь (FIFO). Кладём старт, достаём вершину –
```

```
 у рёбер.",
```

```
 complexity: "O(V + E)",
```

```
 demo: "bfs",
```

```

 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот п
 formal: "push_back / pop_front за $O(1)$. Основа BFS
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся мн
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего
 formal:
 "find со сжатием путей + union по рангу/размеру.
 complexity: "почти $O(1)$ — обратная функция Аккерман
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "t
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
],

```

```

{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "префикс"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от начала»
 formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$. Сумма $a[l..r]$ = $P[r]-P[l-1]$.",
 complexity: "препроцессинг $O(n)$, запрос $O(1)$ ",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sparse-table",
 "разреженная", "двоичные подьёмы", "двоичный подьём",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1, 2, 4, 8, ...,
 м.",
 formal: " $ST[i][j] = op(ST[i][j-1], ST[i+2^{(j-1)}][j-1])$ ",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},

```

```

],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная
 formal: "V-1 ребро, никаких циклов. Краскал (жадно
 но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи
 "амортизированное", "амортизированного", "амортизи
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер
 formal: "Метод потенциалов: дорогая операция оплачи
 ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (су
 ые в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полн
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",

```



```

 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "Z-функция"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся с этой буквы.",
 formal:
 "z[i] = длина наибольшего общего префикса s и s[i..n]",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачи",
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и подзадачи.",
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся подзадачи.",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кратчайшие"],
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» — по стоимости.",
 formal:
 "Дейкстра — неотрицательные веса, O(E log V). Форесман — произвольные веса, O(V^3).",
 demo: "relax",
 simulator: "dijkstra",
}

```

```

 labels: ["планарный", "планарность", "планарного",
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе
 ,
 formal:
 " $V - E + F = 2$ для связного планарного графа. Курс
 ,
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягив
 formal:
 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:

```

```

 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажды
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка O(E log E) + E операций union/find. С
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
 },
],

```

```

 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
 },
 {
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де",
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по случайности. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
 },
 {
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Поиск по ключу.",
 formal:
 "Без балансировки дерево может вырождаться в список.",
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
 },
 {
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "принцип",
 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больше",
 }
}

```

```

 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функции.
demo: "prefix-function",
simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо",
title: "Поворот (rotation) в дереве",
short:
 "Локальная перевеска двух узлов: ребёнок встаёт на
 дившемся месту. Порядок ключей при этом не лом
formal:
 "Правый поворот вокруг p: $x = p.left$; $p.left = x$.
 g и Zig-Zag.",
complexity: "O(1)",
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig — одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в кор
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig — x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: $g \rightarrow p \rightarrow x$ идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб

```

```

content: `<section id="intro-content" class="mb-12
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
},
{
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white":
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2

```

```

{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Форд-Б
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-rose-300 italic m
 <p class="text-xl font-mono text-white"
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 <div class="border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Проверяет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 <div class="border-rose-500 shadow-md">
 Отрицательный цикл
 </div>

```

```

 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb
три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
</div>

 <div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0cr
 <p class="text-lg font-mono text-white">Бор (Tr

```



```

struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по суффиксу
 int term_link = 0; // Терминальная ссылка: мат. слово оканчивается здесь
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные link
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по суффиксу родителя
 nodes[child].link = nodes[nodes[v].link].go[ch];

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 } else {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 }

 q.push(child);
 }
 }
}


```

```

 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2, эйлерова пути нет. Если нечётных ровно 2 нечётные, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней. Для гамильтонова цикла нет простого критерия. Это NP-полная задача, которую не известно, можно ли решить за полиномиальное время."
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет — нужно удалять лишние рёбра"
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если нечётных ровно 2 нечётные или 0."
 }
]

```

```

 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани про
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с $V=7$?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K_5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K_5 : $V=5$, $E=10$, $3V - 6 = 9$. $10 > 9$
 }
]
};

```

```

 stepNote: "Шаг = спуск по вершине v: либо читаем tree, либо считаем tree
 code: `n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; лист"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми нет детей"},
 { name: "m", role: "середина отрезка m = (l + r) // 2"},
],
},
treap: {
 // у страницы нет интерактивного виджета
 variables: [],
},
"splay-tree": {
 vizTitle: "Splay-дерево (Zig / Zig-Zig / Zig-Zag)",
 stepNote: "Шаг = один поворот, поднимающий узел x на уровень корня",
 code: `x = "x" # поднимаемый узел
p = "p" # родитель x
g = "g" # дед x

print("zig: ", x, "↑", p) # шаг 1
print("zig-zig: ", x, "↑", p, "↑", g) # шаг 2
print("zig-zag: ", x, "↗", g) # шаг 3`,
 variables: [
 { name: "x", role: "узел, который поднимаем к корню"},
 { name: "p", role: "родитель узла x перед поворотом"},
 { name: "g", role: "дедушка узла x – нужен в случае zig-zag"},
 { name: "zig / zag", role: "направление поворота: ↑, ↗, ↘"},
],
},
"splay-rotations": {

```

```

],
},
"prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = вычисление S[i][j] по формуле вклю
 code: `n, m = 3, 4 # строк, столбцов
S = [[0] * (m + 1) for _ in range(n + 1)]

for i in range(1, n + 1): # строка
 for j in range(1, m + 1): # столбец
 print(f"S[{i}][{j}] = A + ↑S[{i-1}][{j}] + ←S[{i}][{j-1}]")
 variables: [
 { name: "n", role: "число строк матрицы", range:
 { name: "m", role: "число столбцов матрицы", range:
 { name: "i", role: "индекс текущей строки (таблиц
 { name: "j", role: "индекс текущего столбца (стол
 { name: "S[i][j]", role: "сумма прямоугольника (1
 на каждом шаге" },
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = вызов fibМемо(n): попадание в кэш",
 code: `n = 5 # цель (демо)
memo = {} # кэш подзадач

шаг: fib(n) → memo[n] или fib(n-1) + fib(n-2)
print("fib(", n, ") → мемо", memo, "?")`,
 variables: [
 { name: "n", role: "номер числа Фибоначчи, которое
 { name: "memo[n]", role: "кэш уже посчитанных значений" },
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в кучу",
 code: `a = [7, 3, 5, 1] # куча-массивом

```

```

 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()
print("тарелка:", top, "| первый из очереди:", first)`,
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `adj = {"A": ["B", "C"], "B": ["D"], "C": [],
visited, order = set(), []

stack = ["A"] # BFS: deque + popleft()
while stack:
 v = stack.pop() # шаг: достали вершину
 if v in visited:
 continue
 visited.add(v); order.append(v)
 stack += adj[v] # соседи v → на стек
print(order)`,
 variables: [
 { name: "v", role: "текущая вершина обхода", gang
 { name: "to / u", role: "очередной сосед вершины
 { name: "visited", role: "множество уже посещённых
 { name: "order", role: "порядок, в котором обход
],
},
},

```

```

comp = {} # фаза 2: вершина → № SCC

for i in range(len(order) - 1, -1, -1):
 v = order[i] # шаг: DFS по adjT из v кра
 print(i, "старт из", v, comp)`,
 variables: [
 { name: "v", role: "текущая вершина DFS (первый пр
 { name: "to", role: "сосед по ребру (на 2-м прохо
 { name: "i", role: "индекс прохода по order на вто
 { name: "order", role: "порядок выхода из DFS пер
 { name: "comp", role: "номер компоненты сильной с
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин
 code: `tin, low, timer = {}, {}, 0

v, to = 0, 1 # текущее ребро DFS
tin[v] = tin.get(v, timer); low[v] = tin[v]

шаг: возврат из to
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v] # → v — точка соч
print(f"v={v} to={to} critical={critical}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед: либо ребёнок в DFS-дер
 { name: "tin[v]", role: "время входа в вершину (п
 { name: "low[v]", role: "минимальный tin, достижи
 { name: "i", role: "перебор стартовых вершин для
],
},
"bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

```

```

 variables: [
 { name: "V", role: "число вершин графа", range: "1"},
 { name: "E", role: "число рёбер", range: "K5: 10"},
 { name: "F", role: "число граней в плоской укладке"},
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = выбор ближайшей непосещённой вершины",
 code: `import heapq

dist = {"A": 0} # остальные = ∞
pq = [(0, "A")]

u, v, w = "A", "B", 4 # шаг: ребро u → v веса w
if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация!
 heapq.heappush(pq, (dist[v], v))
print(dist, pq)`
 variables: [
 { name: "u", role: "вершина с минимальным dist среди"},
 { name: "v", role: "сосед вершины u, до которого"},
 { name: "w", role: "вес ребра (u, v)", range: "по"},
 { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) внутри",
 code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
 u, v, w = "S", "A", 5 # шаг: очередное ребро
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
 if i == 1:
 print(f"итерация {i}: {dist}")

```



```

 variables: [
 { name: "h[v]", role: "потенциал вершины – кратчайший путь на Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешиваемое" },
 { name: "d[v]", role: "расстояние Дейкстры в новом графе" },
],
},
"mst-kruskal": {
 vizTitle: "Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C")]
edges.sort() # по весу

parent = {v: v for v in "ABC"} # DSU
def find(x):
 while parent[x] != x:
 x = parent[x]
 return x

mst = []
for w, u, v in edges: # шаг: очередное ребро
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print(mst)\`,
 variables: [
 { name: "n", role: "число вершин; в остов войдёт n-1 вершина" },
 { name: "m", role: "число рёбер, сортируем по весу" },
 { name: "i", role: "ребро сортируем по весу w; рассматриваем ребро i" },
 { name: "parent[v]", role: "DSU: кто представитель компонента" },
 { name: "rank[v]", role: "высота дерева DSU – по правилу" },
],
},
"mst-prim": {
 vizTitle: "Прим",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

j = pi[i - 1] # длина совпадения
while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
if s[i] == s[j]:
 j += 1
pi[i] = j
print(pi)`,
 variables: [
 { name: "n", role: "длина строки s, для которой с"},
 { name: "i", role: "индекс текущего символа – счи"},
 { name: "j", role: "длина текущего наилучшего сов"},
 { name: "π[i]", role: "префикс-функция: длина наи"},
 { name: "ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l",
 code: `s = "abacaba"
n = len(s)
z = [0] * n
l = r = 0 # правый Z-блок [l, r]

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`,
 variables: [
 { name: "n", role: "длина строки s", range: "n = "},
 { name: "i", role: "позиция, для которой считаем"},
 { name: "l, r", role: "правый крайний Z-блок: отр"},
 { name: "z[i]", role: "длина наибольшего общего п"},
],
},
},

```

```

export function buildSyncTemplate(chapterId: string, chapter: Chapter) {
 const sync = getPageSync(chapterId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} • ${sync.vizLink}`
 : `# «${chapterTitle}»\n# демо на странице нет – сводка`;

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}
```

=====

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----

ФАЙЛ 24/85: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМЕР: 10.5 КБ

-----

```
import { Chapter } from '../../types';

export const tickets1to5: Chapter[] = [
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description: "Дерево отрезков – структура данных для эффективной работы с отрезками.",
 category: "Оптимизации и Продвинутое",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Оптимизации и Продвинутое
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">1. Дерево отрезков</h2>
 </div>

 <div class="space-y-8">
 <div class="p-6">
 <div class="text-3xl font-bold text-indigo-600">1.1. Задача</div>
 <p>Дана последовательность чисел a_1, a_2, ..., a_n. Необходимо поддерживать следующие операции:

 Замена значения элемента a_i на x (1 ≤ i ≤ n, 1 ≤ x ≤ 10^9).
 Вычисление суммы элементов на отрезке [l, r] (1 ≤ l ≤ r ≤ n).

 </p>
 </div>
 </div>
`
 }
];
```

```

 <pre class="bg-slate-950 p-4 rounded
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>
\
},
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-white">2. Двумерная разреженная матрица (Sparse Table)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Задача
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дана матрица ST размера $L \times R$ и количество
 <p class="text-xl font-mono text-white">Длина отрезка: 2^k .
 <p class="text-xl font-mono text-white">Длина отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">

```

```
description: "Дерево поиска по ключу и Куча по приоритету",
category: "Продвинутые структуры",
content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Treap
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Treap</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Описание</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дерево поиска по ключу и Куча по приоритету</p>
 <p class="text-xl font-mono text-white">
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 по значению X. Все, кто меньше X улетают в левую кучу, а кто больше X остаются в правой куче.
 </p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вставляем элемент Y в правую кучу (если Y больше, чем у корня, то делаем его новым корнем). Мы просто смотрим на корни: у кого Y меньше, тот и становится новым корнем.
 </p>
 </div>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">

```

```

 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
(Zig, Zig-Zig, Zig-Zag), гарантируя амортиз
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb
вится корнем дерева (VIP-статус). Если ты ч
ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Балансировка чере
 </div>
 <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounde
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate

```

```

</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>`
 }
];

```

```

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-1">
дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
 },
 {
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-py-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-white">15.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">
 1. Инициализация
 </div>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">

```



```

 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг class="bg-slate-900 text-pink-400">k?</code>
 шу себе проходить через вершину k?</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только k=0
 Шаг k=1: Разрешаем пересадки
 e> через путь class="text-indigo-300">k?</code>
 Шаг k=2: Разрешаем пересадки
 нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

 </div>
</div>
</section>`
 },
 {
 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400"

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">

```

```

 <p class="text-xl font-mono text-white">
ается с соседом.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
</div>
</div>
</section>`
 }
};

```

ФАЙЛ 26/85: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```

import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) — Взгляд назад (КМП",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1

```

```

 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>
 ab: Из начала ("a")
 </code>
 abc: Опять мимо. <code>
 abca: 0! Начало ("a"
 </code>
 abcab: Начало ("ab"

 abcabc: Начало ("abc

 abcabcd: "d" всё исп
 "abca" vs "abcd"). Конец "abcd" не совпада

 </div>
</details>

<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;

```

Это самый быстрый способ найти `Z[i]`, считаешь Z-функцию, и там, где `Z[i]`

`<!-- Аналогия 2 -->`

`<div class="bg-slate-900 p-6 rounded-lg`

`<div class="absolute -top-3 left-4 border-rose-500 shadow-md">`

Аналогия 2: LLM Самокопиаст

`<p class="text-slate-300 text-sm mb`

В LLM Z-функция может применять `[i]` (где `<i>i</i>` указывает назад на сама себя.

`<details class="bg-slate-900/40 rounded-lg`

`<summary class="p-4 font-bold text-slate-300 border-slate-700/50">`

Пример вычисления (Скрыто)

`<div class="p-5 text-sm text-slate-300`

`<p>Тот же пример: abcabcd</p>`

`<ul class="list-disc pl-5 space-y-2`

`<li><code>Z[0]</code> – не счита`

`<li><code>Z[1]</code> (bcabcd  
[1] = 0</code></li>`

`<li><code>Z[2]</code> (cabcd  
</li>`

`<li><code>Z[3]</code> (abcd  
? Нет, в начале 'a', тут 'd'). Совпало 3 бу`

`<li><code>Z[4], Z[5], Z[6]</code>`

```

 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
}

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-2">
 пример сортировка чисел). Класс NP – это не-
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ь отличный переводчик на английский (Сведения
 аче В, то В – это NP-Полная (сосредоточен
 </div>
 </div>
 </div>
</div>
</section>`
 }
};

```

ФАЙЛ 27/85: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы"
 },
];

```

---

```
<h3 class="text-xl font-bold text-indigo-400">
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <!-- Аналогия DFS -->
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
```

```
 ♂ DFS (Поиск в глубину) = Жадный
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-2">
```

```
 Что делает: Жадный алгоритм находит ближайшую вершину (например, минимальную по номеру) и пробует другой путь.
```

```
 </p>
```

```
 <ul class="text-xs text-indigo-300">
```

```
 Где используется:
```

```
 Топологическая сортировка (линейное время)
```

```
 Поиск мостов и точек сочленения
```

```
 Сильно связанные компоненты Киркпатрика
```

```
 Поиск циклов и просто проверка связности
```

```

```

```
 </div>
```

```
 <!-- Аналогия BFS -->
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

```
 ♂ BFS (Поиск в ширину) = Волна
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-2">
```

```
 Что делает: Концентрические волны. Работает через Очередь (FIFO). Продвигается от начальной точки.
```

```
 </p>
```

```
 <ul class="text-xs text-emerald-300">
```

```
 Где используется:
```

```
 Кратчайший путь в Невзвешенном графе
```

```
 Алгоритм Ахо-Корасик (сборка слов)
```

```
 Алгоритм Форда-Фалкерсона /
```



```

 </details>
 </div>
 </div>
 </section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">Эта теорема утверждает, что любой планарный граф можно раскрасить 4 цветами.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего 4
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
`

```

```

 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <p class="text-slate-300 text-sm">Макро-вершиной называют компоненту сильной связности (SCC) ориентированного графа, в котором каждая вершина принадлежит ровно одной SCC. SCC — это максимальный набор вершин, между которыми есть взаимные пути.
 </div>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Пример:
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты сильной связности (SCC) ориентированного графа, в котором каждая вершина принадлежит ровно одной SCC. SCC — это максимальный набор вершин, между которыми есть взаимные пути.
 </p>
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>A → B → C → A</code>, то это SCC.
 Если ты добавишь вершину Г и ребро <code>C → G</code>, то SCC останется той же.
 Правило: Если два цикла имеют хотя бы одну общую вершину, то они принадлежат одной SCC.

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 B -> C -> A. Cycle 2: C -> G -> D -> C. The SCC containing C and G is highlighted." data-bbox="100 100 400 400"/>
 </div>
 </div>
 </div>
 </div>
`

```

```

 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
</div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекресток (удаляют вершину) – районы будут
 г>хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400 font-bold">Особ
 <p>
 Для стартовой вершины обхода правил
 то выше неё прыгать некуда). Поэтому для не
 у него больше 1 независимого ребенка
 </p>
 </div>
</details>

```



```

 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

```

```

 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentTin[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);

 if (currentLow[to] >= currentTin[v] && p !== ap.add(v)) {
 recordStep(
 `Найдена точка сочленения: вершина ${v}`,
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это`,
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он`,
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}
}

```





```

<circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
/>
<text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
>
 {node.label}
</text>

{isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}

```

```
 Посещена
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border">
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded ${
 idx === currentStepIndex ? 'bg-rose-500' : 'bg-slate-500'
 }`}
 >
 {step.desc}
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition-all"
 style={{
 width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
 }}
 </div>
 </div>
 </div>
 </div>
</div>
```

```

 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman ford(graph, start'

```

```

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 updated ? ` Релаксация успешна! dist[${edge.u}] = dist[${edge.u}] + edge.w`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v} обновляет ${edge.v}.`,
 activeCodeLine: 9,
 });
 break;
 }
 }
}

if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null, distances: { ...dist },
 log: ` Завершение алгоритма.`,
 activeCodeLine: 11
 });
}

```

```

<div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg b
 <button onClick={() => setHasCycle(false)}
 bg-blue-600 text-white' : 'text-slate-400'>
 <button onClick={() => setHasCycle(true)} c
 r gap-1 ${hasCycle ? 'bg-rose-600 text-white
 </div>
 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-l
 s">⏮ar </button>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (⏮ar {curr
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
 </div>
)}

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=

```

```

 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
checkingNode ? '#7dd3fc' : '#64748b'} strokeWid
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
split(' ')[0]}</text>
 </g>
);
 }
 }
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 })}
 </div>
)}
}
```

```
 </div>
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 30/85: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
```



```

 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не`
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем`
 });
 }
 }
 } else {
 steps.push({
 activeLine: 6,

```

```

 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-100" >
 <div className="flex flex-col md:flex-row items-center" >
 <div>
 <h3 className="text-2xl font-bold text-white" >
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded" >
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведен
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 </button>
 </div>
 </div>
);

```

```

 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
}}
</svg>
</div>

```

```

 </div>
 <div className="flex items-center gap-2 min-h-10" >
 {step.queue.length === 0 ? (

 Нет шагов в очереди

) : (
 step.queue.map(({qId, i}) => (

 {qId}

))
)}
 </div>
 <div className="mt-2 text-sm text-amber-300" >
 {step.logs}
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 31/85: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 },
 bellman: {
 src: bellmanImg,
 alt: 'Добрый Беллман',
 },
 floyd: {
 src: floydImg,
 alt: 'Добрый Флойд',
 },
};

```

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Codecademy
 * Компактный вариант живёт в шапке главы, крупный — по центру
 */
export const ChapterNav: React.FC<Props> = ({ prev, next, index, total, onGo, compact }) => {
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1.5 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 } else {
 return (
 <div className="flex items-center justify-between text-sm">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-4 py-2 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-5 h-5" />
 Назад
 </button>

 {index + 1} из {total} глав

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-4 py-2 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
);
 }
};
```

```
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-1
 </button>
 })
</nav>
});
};
```

---

ФАЙЛ 33/85: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from './hints/TermHint';
import { getViz } from './vizRegistry';
import { ChapterNav } from './ChapterNav';
import { downloadChapterHtml } from '../utils/exportHtml';
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлено название */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```

 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() },
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">

```

```
<p className="mt-6 flex items-start gap-2 text-
 ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-0">

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кнопки

 </p>
```

```
{viz && {
 <section id="chapter-viz" className="mt-8 scroll-mt-12">
 <div className="flex items-center justify-between">
 <h3 className="flex items-center gap-2 text-emerald-400">
 <Sparkles className="w-4 h-4 text-indigo-500"/>
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2">
 <button
 onClick={() => onOpenSimulator(chapterTitle)}
 className="flex items-center gap-1.5 text-emerald-400
 slate-300 hover:text-white hover:border-indigo-500"
 >
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python-компилятор: откройте комментарии"
 className="flex items-center gap-1.5 text-emerald-400
 slate-300 hover:text-white hover:border-indigo-500"
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
 </div>
}
```



```

{ line: " visited[v] = true;", highlight: "normal"
{ line: " tin[v] = low[v] = timer++;", highlight:
{ line: "", highlight: "normal" },
{ line: " for (int to : adj[v]) {", highlight: "no
{ line: " if (to == p) continue;", highlight:
{ line: "", highlight: "normal" },
{ line: " if (visited[to]) {", highlight: "norm
{ line: " low[v] = min(low[v], tin[to]);",
{ line: " } else {", highlight: "normal" },
{ line: " dfs(to, v);", highlight: "normal
{ line: " low[v] = min(low[v], low[to]);",
{ line: "", highlight: "normal" },
{ line: " if (low[to] > tin[v]) {", highlight:
{ line: " // ЭТО МОСТ!", highlight: "n
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },

```

```

{
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1]
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3",
 activeCodeLine: [13,14,15]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2]
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2]
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:3}, stack: [0,1,2,3]
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:3,4:3}, stack: [0,1,2,3,4]
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:1,3:3,4:3,5:3}, stack: [0,1,2,3,4,5]
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
}

```

```

 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: []
 }
],
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 const [dfsAuto, setDfsAuto] = useState(false);
 const dfsTimer = useRef<NodeJS.Timeout | null>(null);

 useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
 }, [dfsAuto]);

 const step = DFS_STEPS[dfsIdx];

 return (
 <div className="space-y-4">

```

```

 (step.activeEdge[0]===e.v && step.active
);
 const activeBack = step.backEdge && {
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
);
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u
 return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
 }}}
{step.currentNode !== null && step.currentN
 const n = GRAPH_NODES.find(x => x.id ===
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} f
>;
 }}())
{GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b
 stroke={cur ? "#fff" : st ? "#34d399"
 strokeWidth={2.5}
 className="transition-all duration-300
 <text x={n.x} y={n.y+3} textAnchor="mid
label}</text>
 {vis && {
 <>
 <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor

```

```

<div className="bg-slate-900/50 rounded-xl bo
 <div className="text-[10px] font-bold text-
 Стек DFS
 {step.s
 </div>
 <div className="flex flex-col-reverse gap-1
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emera
 : "bg-slate-900 text-slate-400 l
 }`} >Вершина {GRAPH_NODES.find(n=>n..
 <span className="text-slate-500 m
 </div>
 })
 }
 </div>
 </div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
</div>
</div>
</div>
</div>
);
}

```

```

 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

```

```

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие расстояния найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,

```

```

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]},
 activeCodeLine: 9,
 });
 }
 }
}

```

```

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!
 activeCodeLine: 11,
});

```

```

 setSteps(simulationSteps);
}, []));

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

const currentStep = steps[currentStepIndex] || null;

```

```

if (!currentStep) return <div className="text-white">

```

```

 </button>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 md:gap-10" >
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/20 text-white font-mono p-6 flex flex-direction-column align-items-center min-h-[400px]" >
 <div className="absolute top-3 left-3 bg-indigo-950/20 text-white font-mono font-bold shadow-sm" >
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10" >
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="10" markerHeight="10" >
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" >
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 { /* Рёбра */ }
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u && currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.u] === vNode;

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';
 })}
 </div>
 </div>
 </div>
 </div>
</div>

```



```

 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVis
 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-1
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill=
 "middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
</g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60
 t-slate-300'
 `}>
 <div className="flex items-center gap

```

```

 <th className="py-2 px-3">Расстояни
 <th className="py-2 px-3">Из (пред.
 </tr>
</thead>
<tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n]
 const prev = currentStep.previous[n]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentN

 return (
 <tr key={n.id} className={`border
 isCurr ? 'bg-indigo-950/60 for
 }`}>
 <td className="py-2.5 px-3 flex
 <span className={`w-2 h-2 rou
 {n.name}
 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 })}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate

```

```

}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,

```

```

 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

```

```

 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:
 }`}
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.leng
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
 </div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 round
 <h4 className="text-xs font-bold uppercase trac
 <div className="flex flex-wrap items-center gap
 {Array.from({ length: targetN }, (, i) => i

```

```

 { /* Tabs */ }
 <div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Eye className="w-4 h-4" />
 Шаг-бай-шаг состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

```

```

 { /* Tab Content */ }
 <div className="min-h-[320px] flex flex-col justi
 { activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl
 <h5 className="font-bold text-white text-sm

 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ п
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl

```

```

 </div>

 {/* Progress Bar */}
 <div className="mt-8 pt-6 border-t border-slate-800" >
 <div className="flex items-center justify-between" >
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
 style={{ width: `${(currentStepIndex + 1) * 100 / steps.length}%` }}
 ></div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 37/85: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },

```

```

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === vId).length;
 const endDegree = C1_EDGES.filter(e => e.v === vId).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
 }
 }
};

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-transparent">
 <svg className="w-full h-[260px] z-10 relative">
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!;
 const v = C1_NODES.find(n => n.id === e.v)!;
 const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`;
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 })}
 </svg>
 </div>
 </div>
 </div>
)

```



```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если ребро, соединяющее две вершины, имеет тот же цвет (например, красный) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево.',
 'Мы перекрашиваем их в синий цвет и делим граф по этому ребру.',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл.',
 'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра.'
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане, добавление этого ребра создаст цикл. Это нарушит структуру дерева.'
 },
 {
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными ребрами?',
 options: [
 'Потому что он написан только для положительных весов.',
 'Он жадный и считает, что каждый следующий шаг приведет к минимальному пути, не учитывая, что отрицательные ребра могут создать более короткий путь.',
 'Отрицательные ребра создают гравитационный коллапс, который останавливает алгоритм.',
 'Дейкстра просто не любил отрицательные балансы на графах.'
],
 correctAnswer: 2,
 explanation: 'Правильно! Алгоритм Дейкстры предполагает, что все ребра имеют неотрицательные веса. Если есть отрицательные ребра, алгоритм может пропустить более короткий путь, который может быть найден позже.'
 }
],
```

```

 correctAnswer: 0,
 explanation: 'Блестяще! CHM (DSU) позволяет за прак
 },
];

```

```

export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<num
 const [selectedOption, setSelectedOption] = useState<
 const [isAnswered, setIsAnswered] = useState<boolean>
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>

```

```

 const currentQ = quizQuestions[currentQIndex];

```

```

 const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };

```

```

 const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
 };

```

```

 const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 };

```

```

return (
 <div className="bg-slate-900/90 border border-slate-
 >
 <div className="flex items-center justify-between"
 <div className="flex items-center gap-3">

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 </div>

 Bonpoc {currentQIndex + 1} / {quizQuestions.length}

 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-white">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;

 let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border border-emerald-500";
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500";
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700";
 }
 }
 return (
 <div className={optionStyle} key={idx}>
 {option}
 </div>
);
 })}
 </div>
 </div>
 </div>
);
}

return (

```

```
 <div className="flex justify-end pt-4 border-t border-gray-200">
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition-colors duration-200"
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border border-slate-900"
 : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? 'Следующий' : 'Завершить'}
 </button>
 </div>
 </div>
);
};
```

---

ФАЙЛ 39/85: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['A 0', 'B 1', 'B 2', 'Г 3', 'Д 4',
```

```

 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром. узлы`,
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]),
 log: `  Улучшение пути [${i}]→[${j}] через пром. узел [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
 }
 }
}

```

```

<div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 text-sm"> Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600')}`}>{isPlaying
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'

```

```

 {/* Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return {
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 }`}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
 currentStep.j === edge.v);
 return {
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

 }
 })}
 </div>
 });
 })}
 </div>
 </div>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div className="flex justify-between items-ce

```

```
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-4 ${
 e === item.line ? 'bg-blue-900/80 text-amber-400' : ''
 }`} >
 {item.line}
 {item.content}
 </div>
)}
 </div>
 </div>
 </div>
</div>
</div>
);
}
```

---

ФАЙЛ 40/85: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```



```

 .stack], visitedNodes: Array.from(vis) }));
 dfs(u);
 }
}

stack.pop();
steps.push({ type: 'backtrack', node: v, desc:
 ueOrStack: [...stack], visitedNodes: Array.fr
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Have
 ray.from(vis)] }));

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `D
 q], visitedNodes: [...Array.from(vis)] }));
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, de
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }
 }
}

```

```

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId)
const isCurrent = (nodeId: string) => step.node === nodeId

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row gap-4">
 <div className="flex-1">
 <div className="bg-slate-900 border border-slate-800 p-4 min-h-[300px]">
 <svg className="w-full h-full max-w-[600px] max-h-[300px]">
 <g>
 <!-- Edges -->
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u)
 const v = nodes.find(n => n.id === e.v)
 // Check if edge is part of the current path
 const edgeTraversed = isVisited(e.v)
 const edgeClass = edgeTraversed ? 'stroke-slate-500' : 'stroke-emerald-500'

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={edgeClass} stroke-width={2} />
)
 })}

 <!-- Nodes -->
 {nodes.map((n) => (
 <div key={n.id} style={{position: 'absolute', left: n.x, top: n.y}}>
 <div style="width: 10px; height: 10px; background-color: #334d5b; border: 1px solid #334d5b; margin: 2px; display: flex; align-items: center; justify-content: center; font-size: 8px; color: white; border-radius: 50%;">
 {n.id}
 </div>
 <div style="position: absolute; left: 15px; top: 50%; transform: translateY(-50%); font-size: 10px; color: white; font-weight: bold;">{n.name}</div>
 </div>
))}
 </g>
 </svg>
 </div>
 </div>
 </div>
 </div>
)

```

```

 </svg>
 </div>

 { /* Structure View (Stack / Queue) */ }
 <div className="bg-slate-900 border border-
r">
 <div className={`absolute -top-3 left
 ${mode === 'dfs' ? 'border-
ld-300 border border-emerald-500'}`}>
 {mode === 'dfs' ? 'Стек вызовов' : ''}
 </div>

 <div className="flex flex-col gap-2 w-full" style={{
 {step.queueOrStack && step.queueOrStack.length > 0}
 }}>
 <div key={idx} className={`w-full flex items-center justify-between
 ${mode === 'dfs' ? 'bg-slate-800 text-emerald-200' : 'bg-slate-900 text-white'}
 ${idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? 'border-0_10px_rgba(245,158,11,0.2)'} : ''}
 `}>
 {mode === 'dfs' ? 'Вызван: {item}' : '{item}'}
 {idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? Закрыть
 : ''}
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-2" style={{background-color: 'black', color: 'white', padding: '10px'}}>
 { /* Controls */ }
 <div className="flex bg-slate-900 border border-slate-800 rounded p-2 text-slate-400 hover:text-white hover:background-color:slate-800" style={{width: '100%', gap: '10px'}}>
 <button onClick={() => {setAutoPlay(!autoPlay)}}>{autoPlay ? 'Пауза' : 'Воспроизведение'}</button>
 <button onClick={() => {setStep(step + 1)}}>Следующий шаг</button>
 <button onClick={() => {setStep(0)}}>Сброс</button>
 </div>
 </div>

```

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 }
 }
}
```

```

 { name: 'Пётр (Кашель)', symptom: ' Простуда', emo
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS
 });
 return arr;
})());

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string[]>([]);
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({

```

```
 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);
```

```
 // --- INSERT: Новый пациент поступает в приемный покой
```

```

 const insertPatient = useCallback((name: string, prio
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выпис
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

```

```

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const finalName = name || preset.name.split(' ')[0];
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

```

```

 const newPatient: Patient = {
 id: `p${uid++}`,

```

```

 setShake(true);
 addLog(`⚠ Нет пациентов для экстренной госпитализации`);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'winning' } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exiting' } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animState: 'sifting' }));
 });

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
 addLog(`Успех! Пациент ${topPatient.name} перенес операцию`);
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);
 }, 800);
 }, 800);

```

```

 });

 return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Priority Queue</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В реанимации не важно, кто пришел раньше, а кто в
 настоящее время (максимальный приоритет).
 Куча (Heap) автоматически перестраивает папку,
 оказавшаяся на самом вершине дерева (в корне).
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex flex-col lg:flex-row items-stretch gap-6">
 { /* Добавить случайного */ }
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-700
 opacity-40 disabled:cursor-not-allowed text-emerald-50 font-medium
 py-2.5 ve:scale-95 transition-all cursor-pointer focus:outline-none"
 >
 Привезти по Скорой (INSERT)
 </button>

 { /* Добавить кастомного */ }
 <form onSubmit={handleCustomInsert} className="flex-1">
 <input
 type="text"
 required
 />
 </form>
 </div>
 </div>
);
 }
}

```

```

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

```

```

 { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
 <div className="grid grid-cols-1 lg:grid-cols-12" >

```

```

 { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
 -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
 <div className="absolute top-4 left-4 text-[14px] font-medium" >
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

 <div className="relative w-full h-full min-h-[380px]" >
 <svg className="absolute inset-0 w-full h-full" >
 {edges}
 </svg>

```

```

 {heap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center justify-center
 -translate-x-1/2 -translate-y-1/2 transition-all
 ${patient.animState === 'in' ? 'animate-in' : ''}
 ${patient.animState === 'winner' ? 'animate-winner' : ''}
 ${patient.animState === 'out' ? 'animate-out' : ''}
 ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
 `

```



```


 </div>
 </div>
 <div>
 <h4 className="text-white font-black">
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-x-small">
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-4">

 <Heart className="w-4 h-4 text-rose-500" />

 </div>
 </div>
</div>
) : (
 <div className="text-center text-slate-600">

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Ждём самовосстановления
 </p>
 </div>
 </div>
)
</div>

<div className="w-full h-3 bg-gradient-to-r from-rose-500 to-rose-400">
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {entry.action}
 {entry.time}
 </div>
))}
 </div>
</div>

```

```

const paused = useContext(DemoPauseContext);
const period = Math.round(ms * SPEED);

useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1) * 1000), period);
 return () => clearInterval(t);
}, [steps, period, paused]);

return step;
}

export const C = {
 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза — курсор на карточке" : "наведи"}
 </div>
 </div>
);

```

```

 style={{ transition: MOVE }}
 />
);

```

ФАЙЛ 43/85: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 9], j: [9, 4], k: [3, 3] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.done} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.bad} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.bad} />

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=

```

```

 return <Node key={k} x={x} y={y} r={12} label={
 }
 </Svg>
);
};

```

/\*\* Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50] };
 const edges: [string, string, number][] = [
 ["A", "B", 1],
];
 return (
 <Svg
 caption={
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
 }
 >
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive}>
 {step >= 2 && (
 <text
 x={({a[0] + b[0]}) / 2}
 y={({a[1] + b[1]}) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
 })}
 </Svg>
);
};

```

```

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={14}>
 <text x={53 + i * 52} y={77} textAnchor="middle">
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={14}>
 <text x={startX + i * (boxW + 4) + boxW / 2} y={112}>
 {sorted.indexOf(v)}
 </text>
 </g>
 })}
 </Svg>
);
};

```

/\* ----- Splay: три случая поворота ----- \*/

```
type Pt = [number, number];
```

```

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

 return (

```

```

<SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В
 frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
/>
);

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, x вправо", "Поворот НИ
 frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
);

```

ФАЙЛ 44/85: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

```

```

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["носки", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, { x: n.x, y: n.y }]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `}
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
))}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={-10} dy={-10}>{n.id}</text>
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={10} />
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle">{placed[idx]}</text>
)}
 </g>
);
 })}
 <text x={224} y={62} textAnchor="middle" fontSize={16}>Порядок:</text>
 <text x={224} y={74} textAnchor="middle" fontSize={16}>{placed.join(" → ") || "..."}</text>
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {

```

```
export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [["A", "B"], ["A",
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные
 {edges.map(([u, v], i) => {cut && (u === "C" || v
 {Object.entries(pos).map([k, [x, y]]) =>
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="no
) : (
 <Node key={k} x={x} y={y} label={k} fill={k =
 } />
)
)}
 </Svg>
);
};
```

```
export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
```



```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit";

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине">
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
))}
 </Svg>
);
};
```

```
export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"], ["A", "B", "C", "D", "E"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ часть стека" : "push: добавляем элемент в стек"}>
 <rect x={90} y={22} width={80} height={96} rx={6} />
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26} />
 <text x={130} y={111 - i * 26} textAnchor="middle" />
 </g>
))}
 <text x={130} y={16} textAnchor="middle" />
 </Svg>
);
};
```

```

 par !== i ? {
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
 fill="none" stroke={rootColor[root(i)]} stroke-width={strokeWidth}
 } : null
 })
 {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}
 </Svg>
);
};

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий путь = префикс"
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]}
 })}
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x} y={n.y}
 </Svg>
);
};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых"
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.dim} />
 <text x={n.x} y={n.y} textAnchor="middle" dom
 fill={C.dim} />
 </g>
))}
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` : "о префиксные суммы P"}
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={16}
 fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.dim} />
 <text x={20 + i * 39 + 17} y={24} fill={C.dim}>{v}</text>
 </g>
))}
 </Svg>
);
};

```

```
export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
)}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done}
 <text x={130} y={20} textAnchor="middle" fontSize={14}
 </Svg>
);
};
```

```
export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done}
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333"
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
 <text x={206} y={34} textAnchor="middle" fontSize={14}
 <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.bad}
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
 ?"></text>
 <text x={130} y={124} textAnchor="middle" fontSize={14}
 </Svg>
);
};
```

```
export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
```

```

 fontSize={10} fontWeight="700" fill="black"
)}
 </g>
);
})
})
<text x={130} y={122} textAnchor="middle" fontSize={10}>
 каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

ФАЙЛ 46/85: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components"
 | "zig" | "zig-zig" | "zig-zag";

```

```
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};
```

---

ФАЙЛ 47/85: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
 vizId: string | null;
 onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
```

```
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 };
 });
```

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```
interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
```

```
 useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
```

```

 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
>
<div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >Подсказка
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300">
 {entry.demo && <MiniDemo kind={entry.demo} />}

 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300">
 {entry.formal}
 </p>
)}

 {entry.complexity && (
 <div className="text-[11px] font-mono text-slate-300">
 Сложность: {entry.complexity}
 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 >

```



```

 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 1 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 2 },
];

```

```

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

```

```

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4 4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4 4";
 return "stroke-slate-700 stroke-dasharray-4 4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4 4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4 4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4 4";
 if (step > 6) return "stroke-emerald-500";
 }
};

```

```

 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1
 return e.newWeight;
 }
 return e.weight;
 });

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
 ми.
 Это наша база для расчета потенциал
 </div>
);
 }
};

```

```

 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
шин!
 </div>
);
 default: return "";
 }
 };
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border"
 <h4 className="text-sm md:text-base font-bo
 <span className="w-2.5 h-2.5 bg-amber-400"
 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row gap-4"
 <div className="bg-[#0f172a] flex-1 min-h-100
 nter p-4">
 <svg width="400" height="300" class="w-full h-full"
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5"
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5"
 tart-reverse">
 <path d="M 0 0 L 10 5 L 10 0" />
 </marker>
 </defs>
 <g>
 <use href="#arrow-slate" x="100" y="100" />
 <use href="#arrow-slate-dim" x="100" y="100" />
 <use href="#arrow-amber" x="100" y="100" />
 <use href="#arrow-pink" x="100" y="100" />
 </g>
 </div>
 </div>
 </div>
);
);

```

```

 `}>
 {getEdgeContent
 </text>
 </g>
)
}}}

{/* Nodes */}
{nodes.map(n => {
 if (!isNodeVisible(n.id)) r
 const pot = getPotential(n.

 return (
 <g key={n.id} className=
 <circle
 cx={n.x} cy={n.y}
 className={`str
 ${n.id === 'S'
 ? 'fill-[#2
 : 'fill-sla

 />
 <text x={n.x} y={n.y}
- white">
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)}
 </g>

```

```
 125' : i < step ? 'bg-indigo-900' : 'bg-slate-900'
)))
 </div>
 </div>
</div>
</div>
);
}
```

---

ФАЙЛ 50/85: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
];
```

```

const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(

```

```

);

 // Transpose adj
 const adjT: number[][] = Array.from({ length: 5 },
 initialEdges.forEach((e) => adjT[e.v].push(e.u));

 // Reset visited for phase 2
 visited.clear();
 let sccId = 0;

 const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
 };

 // Run DFS2 using the order (reversed from back to front)
 for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
);
 }
 }
}

```

```

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >

```



```

 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#66bb6a" />
 </marker>
</defs>

```

```

{/* Edges */}
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isScEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

```

```

const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentN
const sccId = step.sccMap[node.id];

let classes = "fill-slate-800 stroke-slate
if (sccId) {
 classes = getSccColor(sccId);
} else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-
} else if (isVisited && step.phase === "d
 classes = "fill-emerald-700 stroke-emera
}

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text

```

```

<div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-500" >
 {step.order.length === 0 ? (

 Нет элементов в стеке

) : (
 [...step.order].reverse().map((val, idx) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-slate-500 p-1.5 rounded"
 >
 {val}
 </div>
 })
)}
 </div>
</div>

```

```

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px] bg-slate-500" >
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded ${s.type === 'error' ? 'bg-red-900 text-white' : 'bg-slate-500 text-slate-300'}`}
 >
 {s.desc}
 </div>
 })
 }
 </div>

```

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Checkmark } from 'react-native';
import { useNavigation } from '@react-navigation/native';

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
},
// Step 2: Include A-B
() => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)

```

```
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
```

```

 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветную H',
 type: 'info'
 }, ...prev]));
 },
 // Step 18: Include G-H
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в остове',
 description: 'Вершина H перекрашена в фиолетовую',
 type: 'success'
 }, ...prev]));
 },
 // Step 19: Inspect E-H (11) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединение создаст цикл!',
 type: 'warning'
 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {

```

```

 description: 'Плесень тянется по ребру D-E к ве
 type: 'info'
 }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вер
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)

```



```

 type: 'success'
 }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]);
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{

```

```

 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
 },
 // Step 18: Pop G-H (10)
 () => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
 },
 // Step 19: Include G-H, add H's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
 },
 // Step 20: Pop E-H (11) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]));
 },
 // Step 21: Reject E-H

```

```

// Node 1(B) -> A-B(2)
// Node 2(C) -> B-C(4)
// Node 3(D) -> D-E(3)
// Node 4(E) -> D-E(3)
// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы

```

```

 type: 'success'
 }, ...prev]);
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин гра
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 }
};

```

```

 return '#475569';
 };

 return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Супер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white">
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>

```

```

 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
 { /* Graph Canvas */ }
 <div className="lg:col-span-2 bg-slate-950 rounded
 active min-h-[480px] overflow-hidden">

 { /* Legend */ }
 <div className="absolute top-4 left-4 bg-slate-900
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white" />
 </div>
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white" />

```

```

 const u = vertices[edge.u];
 const v = vertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }
 }
 </svg>

```

```

 </h4>
 </div>

 {activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 border-1"
 <p className="text-[11px] font-bold text-white">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? {

 Пусто

 } : {
 heapQueue.map(({item, idx}) => {
 <span key={idx} className={["px-2 py-1",
 "border border-slate-700 shadow-md animate-pulse" : "bg-slate-800 border-slate-700"]
 .join(" ")}>
 {item}

 })
 })
 </div>
 </div>
)}

 <div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all"
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500'
 : 'bg-slate-900/60 border-slate-700')
 >
 {log.message}
 </div>
 })}
 </div>
}

```



```

 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-rose-600")
 }
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

 {activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style={{ gap: 10px }}>
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Крускал</p>
 <p className="text-slate-400 text-xs mt-1">Алгоритм нахождения минимального остовного дерева</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs mt-1">Алгоритм нахождения минимального остовного дерева</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Код</p>
 <p className="text-slate-400 text-xs mt-1">Алгоритм нахождения минимального остовного дерева</p>
 </div>
 </div>
 </div>
)}
 </div>
</div>

```

```

<div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Award className="w-3.5 h-3.5 text-em
 </p>
 <p className="text-2xl font-black text-1
 <p className="text-slate-400 text-xs mt
</div>
<div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
положении:</p>
 <p className="text-xs text-slate-300 le
ие чистые вершины через самое дешевое досту
 </div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Code className="w-3.5 h-3.5 text-eme
 </p>
 <pre className="text-xs text-emerald-400
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
```



```
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110 transition duration-300"/>
 </div>
 <div>
 <h3 className="text-lg font-bold mb-1 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">
 {c.title}
 </h3>
 <p className="text-slate-400 text-sm">
 {c.desc}
 {c.status}

 </p>
 <div className={`mt-3 text-xs font-mono px-2 ${c.isCurrent ? 'text-white' : 'text-slate-400'}>
 {c.code}
 </div>
 </div>
 </div>
))}
 </div>
);
}
```

---

ФАЙЛ 53/85: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";
```

```
interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что...
```

```

 Содержание · {index + 1} из {total}

 <span className="block text-[13px] font-bol

 </button>
</div>

{open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950"
 <div className="absolute inset-y-0 left-0 w-[
 te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-b
 <span className="font-bold text-white tex
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hov
 aria-label="Закреть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapte
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
)}
</>
);
};
```

---

```
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-slate-800">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={() => setActiveTab('compiler')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Компилятор
 </button>
 </div>
</div>
```

```
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
```

---

ФАЙЛ 55/85: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
```

```

 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
 }
 return <>{lines}{circles}</>;
 })()}
</svg>
<div className="absolute bottom-2 left-2 right-
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 })}
 </svg>
</div>

```



}

---

ФАЙЛ 56/85: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
```

```
 ArrowRightLeft,
```

```
 Check,
```

```
 CheckCircle2,
```

```
 Copy,
```

```
 HelpCircle,
```

```
 Link2,
```

```
 Loader2,
```

```
 Play,
```

```
 Plus,
```

```
 RotateCcw,
```

```
 Terminal,
```

```
 Unlink,
```

```
 Variable,
```

```
 X,
```

```
} from "lucide-react";
```

```
import { buildSyncTemplate, getPageSync } from "../data";
```

```
import { Tooltip } from "../Tooltip";
```

```
const PYODIDE_VERSION = "0.27.7";
```

```
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7`;
```

```
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;
```

```
interface PythonCompilerProps {
```

```
 /** Страница, с которой синхронизируется редактор. */
```

```
 chapterId: string;
```

```
 chapterTitle: string;
```

```
 /** Перейти к странице пособия (посмотреть визуализацию кода) */
```

```
 onOpenGuide: () => void;
```

```

/* «Вставить» кладёт сниппет в редактор под курсор.
/* -----

interface Snippet {
 label: string;
 tip: string;
 code: string;
}

const SNIPPETS: { group: string; items: Snippet[] }[] =
 [
 {
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (3, "C", "A")]\nedges\n',
 },
],
 },
 {
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы
 code: "n, m = 3, 4\nfor i in range(n):
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация"
```

```

 },
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}]",',
 },
],
},
];

```

```

/** Переживает сккрытие панели: код пользователя не теряется
let savedEditor: { code: string; template: string; chapterId: string; } = {

```

```

export function PythonCompiler({ chapterId, chapterTitle, chapterContent }) {
 const sync = useMemo(() => getPageSync(chapterId), [chapterId]);
 const template = useMemo(() => buildSyncTemplate(chapterContent), [chapterContent]);

```

```

 const [code, setCode] = useState(() => {
 if (!savedEditor) return template;
 if (savedEditor.code === savedEditor.template || savedEditor.code === "")
 return savedEditor.code;
 });

```

```

 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "highlight" | "none">("vars");
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);

```

```

 const [desyncedFrom, setDesyncedFrom] = useState<string | null>(null);

```

```

 setRunning(true);
 setOutputOpen(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." :

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = stdin.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется

 runtime.setStdout({ batched: (message) => stdout.push(message)});
 runtime.setStderr({ batched: (message) => stderr.push(message)});
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null)});
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вывела");
} catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить — проверь код");
} finally {
 setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать — буфер переполнен");
 }
});

```



```

 hasViz ? "text-emerald-400" : "text-slate-400"
 }` }
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визита"}

</Tooltip>
</div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-w-300">
 <Tooltip side="bottom" content="Открыть эскиз">
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center gap-1.5 px-1.5 py-1.5 text-slate-300 hover:text-white hover:bg-slate-700">
 {chapterTitle}
 </button>
 </Tooltip>
 </div>
 {desynchedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница была заменена."
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5 px-1.5 py-1.5 text-amber-400 hover:bg-amber-500/20 hover:text-white">
 <Unlink className="w-3.5 h-3.5 shrink-0">
 Страница сменилась – подставить её по умолчанию
 </Unlink>
 </button>
 </Tooltip>
)}
 </div>
) : null}

```

```

 </Tooltip>
 })
 </div>
 })
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map(({gItem}) => {
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map(({s}) => {
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-s
 {s.label}
 </button>
 </Tooltip>
)}}
 </div>
 </div>
)}}
 </div>
 <p className="text-[10px] text-slate-600">H
</div>
})
</div>

```

```

{/* — Редактор + ввод
<div className="flex-1 min-h-0 flex flex-col">
 <textarea
 ref={editorRef}
 value={code}

```

```
 <Terminal className="w-3.5 h-3.5 text-emera

 <span className="inline-flex items-center gap
 {runtimeReady && <CheckCircle2 className="w
 {outputOpen

 </button>
 {outputOpen && (
 <>
 <pre className="flex-1 min-h-[70px] overflow
 x] leading-5 text-slate-300">
 {output}
 </pre>
 <div className="px-3 py-1.5 border-t border
 </>
)}
</div>
</aside>
);
}
```

---

ФАЙЛ 57/85: `src/components/QueueViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```



```

 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже.
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
 }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${h

```

```

{ /* МНЕМОНИКА / ПРАВИЛО */ }
<div className="bg-indigo-950/40 border border-in
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 tex
 <p className="text-xs text-slate-300 mt-1 lea
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата – только строгий п
 </p>
 </div>
</div>

{ /* УПРАВЛЕНИЕ */ }
<div className="flex items-center gap-3 flex-wrap
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-t
 d:opacity-40 disabled:cursor-not-allowed te
 ive:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-8

```

```

<div className="flex flex-col items-center" style={styles.container} >
 Загрузка...
 Список очереди
</div>

{/* СПИСОК ОЧЕРЕДИ */}
<div className="flex-1 flex items-end gap-4" >
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col justify-center align-center" >
 Ожидание
 <p className="text-xs font-bold" >Ваша очередь пуста</p>
 <p className="text-[10px]" >Все сыты и довольны</p>
 </div>
) : (
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
 return (
 <div
 key={customer.id}
 className={`flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `
 >
 {/* Облачко мыслей */}
 <div className={`absolute -top-16 left-1/2 -transform translate-x-50%
 text-center text-slate-200 leading-tight font-size-8
 ${isHead ? 'opacity-100 border' : 'opacity-0'}
 `}>
 <div className="font-bold text-slate-200" >{customer.bubbleText}</div>
 <div className="absolute -bottom-16 left-1/2 -transform translate-x-50%
 font-size-8 text-center text-slate-200
 it rotate-45"></div>
 </div>
 >
)
 })
)}
</div>

```

```

<div className="absolute top-4 right-4 flex items-center justify-center"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 СЫТЫ
</div>

<div className="flex-1 flex flex-col justify-between"
 {servedHistory.length === 0 ? {
 <div className="flex-1 flex flex-col item-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </p>
 </div>
) : {
 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded"
 >
 {item}
 </div>
 })}
 </div>
 })}
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => {

```

```

const [newWord, setNewWord] = useState('');
const [nodes, setNodes] = useState<{ [key: number]: T
const [simulationStep, setSimulationStep] = useState<
const [bfsQueue, setBfsQueue] = useState<number[]>([])
const [currentNode, setCurrentNode] = useState<number>
const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и буль!')
);
const [isTalking, setIsTalking] = useState<boolean>(false);
const [isBlinking, setIsBlinking] = useState<boolean>(false);

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false), 1000);
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {

```

```

 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : rightX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

```

```

 ь полюбоваться таблицей переходов!'
);
 return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.char}`;

for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
}

if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);

```

```

<svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_10px]_black' : ''
 }`
>
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85" fill="#e67e22" />

 {/* Хвост */}
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-pulse"
 />

 {/* Плавники */}
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#f87171" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#f87171" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#fff" />
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" stroke="black" />
) : (
 <circle cx="148" cy="85" r="6" fill="#fff" />
 <circle cx="151" cy="82" r="2" fill="black" />
)}

 {/* Рот (анимация разговора) */}

```



```

 <Info className="w-3.5 h-3.5 text-blue-400">
 Статус: {simulationStep === 0 ? 'Бор готов к бою!' : 'Бор проиграл!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-2 py-1 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS
 1
 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 text-white px-2 py-1 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>
 </div>
</div>
</div>

```



```

 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 </div>
 </div>

```

```

 </form>
 </div>

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-slate-700">
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 }`}
 >

```

```

 {node.id === 0 || node.term_link
 <span className="text-slate-500"
) : (
 <span className="bg-rose-950 text-white
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

)}
 </td>
 <td className="py-2.5 px-3 text-slate-500"
 {Object.keys(node.children).length === 0
 ? '0'
 : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? (
 <span className="text-slate-600"
) : (
 <span className="bg-emerald-900 text-white"
 {node.words.join(', ')}

)}
 </td>
 </tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
 void CODE;

 function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
 { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
 }
}
```

```
 return steps;
}
```

```
// 3. Bottom-Up Query steps (iterative on implicit tree
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

 while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 4, treeState: tree, highlightNodes: [l], arrayHighlights: highlights });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный → r`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlights: highlights });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 }
}
```

```

 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' } };

```



```

 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full"
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 /* Array strip */
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900
 <span className="text-[7px] text-slate-50
 <span className="text-[11px] font-bold fo
 </div>
);
)}}
 </div>

 /* Tree */
 <div className="flex-1 p-2 overflow-x-auto bg-sla

```

```

 0 hover:text-white hover:bg-slate-800 rounded
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-400'}
 {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}
 .5 text-slate-400 hover:text-white hover:bg-slate-800 rounded px-1.5 py-1 outline
 </button>
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))}
 text-[9px] font-bold rounded px-1.5 py-1 outline
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className="bg-rose-500" style={{ width: ((current + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8, 3, 12');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);
}

```

```

 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2 transition-colors">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 A[{i}]

))}
 Сумма
 </div>
 </form>

 {/** Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500 focus:border-indigo-500">
 <label className="text-[10px] font-bold uppercase">Запрос</label>
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 От
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1">
 До
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />
 </div>
 </div>
 </div>
)}

```

```

{view === 'build' && (
 <SimPanel
 key={`build-${arr.join(' - ')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↑ "
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 </div>
)}

```



```

<div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute" />
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-indigo-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-indigo-500"
 aria-label="Очистить поиск"
 />
 <X className="w-3.5 h-3.5" />
)}
</div>
</div>

```

```

<div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm text-slate-500">Нет тем</p>}
 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 />
)
 })}
 </div>
 </div>
)}
</div>

```

```
 </aside>
);
};
```

---

ФАЙЛ 61/85: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```
export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |
```

```
 // Stack state
```

```
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' }
```

```

 setPopMessage(` Положили поверх стопки: ${newPlate}
 });

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate}
});

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson}

```



```

 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
`);

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 p-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-slate-400 to-slate-600">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>
 </div>

```

```

 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 te
 : 'bg-slate-800/60 border-slate-700/60 te
 }`>
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

 </button>
 </div>

 {/* STACK TAB */}
 {activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>

```

```

{stack.length === 0 ? (
 <div className="text-center py-12 text-slate-500">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стол пуст</p>
 <p className="text-xs mt-1">Добавьте тарелку</p>
 </div>
) : (
 <div className="w-full max-w-sm flex flex-direction-column align-items-center">
 <div className="absolute top-0 text-xs font-mono text-right">
 full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 animate-bounce">
 </div>

 {/* Reversed map so top of stack is visible */}
 {stack.slice().reverse().map((plate, index) => {
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-center justify-center gap-1
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-100'
 : 'bg-slate-900/90 border-slate-500'}
 `}
 >
 <div className="flex items-center gap-1">
 {plate.name}
 <span className={`font-bold text-2xl`}
 >{plate.name}
 </div>
 <div>
 <span className={`text-xs font-mono`}
 >{isTop ? 'Готово к помывке' : 'В процессе мытья'}
 </div>
 </div>
);
 })}
 </div>
 </div>
);

```

```

er border-indigo-500/40 px-4 py-2.5 rounded
>
 Налить суп первому
</button>
<button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
 <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{dequeueMessage && {
 <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
 } : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">

 <span className="text-xs font-bold fo

```

```

 -600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
 </div>
 })
 <div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
 </div>
 </div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}

```

```

 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center
 emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в очередь
 </button>
 </div>
 </form>

```

```

 {heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/60
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rounded-full" style={{
 background-color: heapMessage.color
 }} />
 {heapMessage}
 </div>
)}

```

```

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl" >
 <div >
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча пуста</p>
 <p className="text-xs mt-1">Добавьте элемент</p>
 </div>
) : (
 <div className="w-full max-w-2xl space-y-4">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>
 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;

```



```

<Search className="w-4 h-4 text-slate-500 absolute
<input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500
/>
{query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
</div>

{items.length === 0 && <p className="text-slate-500">
 Нет результатов
</p>}

<div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 border border-
 >
 <h3 className="font-bold text-white text-sm">{chapterTitle}</h3>
 {entry.hint && <p className="text-[12px] text-white">{entry.hint}</p>
 && <div className="flex-1" />)}
 <div className="flex items-center gap-2 mt-2">
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 text-white transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Запустить
 </button>
 </div>
 </div>
)}
</div>

```



```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",
 hintCase: "Zig",
```

```

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
 }

 /** Какой поворот сделал бы настоящий splay из текущего
 export function canonicalNext(root: TNode, key: number) {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя поворот)" };
 }

 /** ----- раскладка -----

 interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
 }

```

```

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)));

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;

```

```
viewBox={`0 0 ${width} ${height}`}
className="h-auto block mx-auto"
style={{ minWidth: Math.min(width * 1.5, 420)
role="img"
```

>

```
{edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-b
 />
);
}}}
```

```
{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}
 transition: "transform 700ms cubic-be
 cursor: clickable ? "pointer" : "defa
 }}
 >
 {isTarget && (
```

```

 : "bg-slate-800 border-slate-700 text-slate-500"
 }` }
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : null}
 {showAnswer ? "Скрыть разбор" : "Сдаться / показать"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

</div>

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

{solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"
 }` }
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : null}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`}
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (

```

```
* идёт в своём темпе. Автопрокрутка включается кноп
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}
```

```
const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,
```

```

 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит"
 }
]
};

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p: [148, 112] },
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"], ["p", "C"]],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 }
]
}

```



```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
)}
)}
 </svg>
)

```

```

 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke=
 <circle r={r} fill={c.fill} stroke={c.stroke}
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].p
 const duration = useMemo(() => Math.round(current.ms
 const last = idx === total - 1;

```

```

 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }` }
 >
 {current.kind === "aim" ? "прицел" : current.title}

 Шаг {idx + 1} из {total}. {current.title}

</div>
<p className="text-[13px] text-slate-400 leading-tight">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 font-mono text-sm">
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-sm
 ${/\d/.test(t) ? "bg-emerald-500/15 text-white" : ""}
 `}
 >
 {t}

))}

 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

```

```

 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-3
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-600"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center justify-center"
 <span className="w-3 h-3 rounded-full"
 {active[id]} />

 {active[id]}
)
)}
 </div>
</div>

```

```
// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi
 // Values: 10, 20, 30, 40, 50, 60
 const createInitialTree = (): SplayNode => {
 let r: SplayNode | null = null;
 const initialKeys = [40, 20, 50, 10, 30, 60];
 initialKeys.forEach((key) => {
```

```

 }
 }

 const targetKey = current ? key : lastNode.key;
 const isFound = !!current;

 if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И
);
 } else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем
);
 }

 // Now let's implement standard Splay logic recursively
 const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right
 if (node.left.right) {

```

```

 return { newRoot: finalRoot, steps };
 };

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree, val);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

```

```

 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 data-title={`Кликните, чтобы выполнить Splay(${
 >
 <circle
 cx={node.x}

```



```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-12" style={{width: 1000px; height: 1000px; position: relative; border: 1px solid #ccc; background-color: #f0f0f0; margin: 10px 0; border-radius: 10px;}}>
 {/* Playfield */}
```

```
 <div className="lg:col-span-7 bg-[#0f172a] p-4" style={{width: 100%; height: 100%; position: relative; border: 1px solid #ccc; background-color: #0f172a; color: white; margin: 10px 0; border-radius: 10px;}}>
 <svg
 className="w-full h-full max-w-[600px] max-h-[320px]"
 viewBox="0 0 600 320"
```

```
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>
```

```
 {/* Quick interactive controls */}
```

```
 <div className="absolute bottom-4 left-4 right-4" style={{width: 100%; height: 100%; position: relative; border: 1px solid #ccc; background-color: #f0f0f0; margin: 10px 0; border-radius: 10px;}}>
 <form onSubmit={handleFind} className="flex" style={{width: 100%; height: 100%; position: relative; border: 1px solid #ccc; background-color: #f0f0f0; margin: 10px 0; border-radius: 10px;}}>
```

```
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 className="w-24 bg-slate-950 text-white"
 style={{width: 100%; height: 100%; position: relative; border: 1px solid #ccc; background-color: #f0f0f0; margin: 10px 0; border-radius: 10px;}}
 00 focus:outline-none"
 />
```

```
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo-700"
 style={{width: 100%; height: 100%; position: relative; border: 1px solid #ccc; background-color: #f0f0f0; margin: 10px 0; border-radius: 10px;}}
 >
```

```
 Поиск / Splay
```

```
 </button>
```

```
 <button
```

```
 type="button"
 onClick={(e) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (!isNaN(val)) {
 let newTree = cloneTree(tree);
```

```

 }` }
 >
 {step}
 </div>
 }}}
</div>

<div className="border-t border-slate-800 pt-2">
 <p>

 у самого корня. Делаем 1 вращение.
 </p>
 <p>

 оба левые или оба правые. Вращаем деда, по
 </p>
 <p>

 колено. Вращаем узел в разные стороны.
 </p>
 </div>
 </div>
 </div>

 /* Answers Section inside the visualizer widget
 <div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-slate-300">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нитья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3">
 <div className="bg-slate-900/60 p-4 rounded-lg">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>
 <p className="text-slate-400 leading-relaxed">

```

```

 </div>
 </div>

 <div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb
 3. Амортизация $O(\log n)$
 </p>
 <p className="text-slate-400 leading-rela
 Парадокс: один запрос за{" "}
 <code className="text-emerald-400"> $O(n)$ </code>
 всю структуру вдвое более плоской за сч
 при двойных вращениях {
 <code className="text-emerald-400 font-
 }. Дорогая операция инвестирует в дешев
 операций. Поэтому амортизированное врем
 <strong className="text-white"> $O(\log n)$
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emera
 Каждое "растяжение" компенсируется "склад
 </div>
 </div>
 </div>
</div>
);
};

```

---

ФАЙЛ 66/85: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```
// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }
}
```

```

 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

 const topIndex = dirtyStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 transition-all cursor-pointer flex items-center justify-center">
 Сбросить (POP)
 </button>
 </div>
 </div>
);
}

```

```

"></div>
 <div className="absolute text-4xl anim-cr
</div>
})

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map(({plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
 hadow-md select-none transition-all duration
 ${plate.color}

```

```

 <p className="text-xs font-bold font-mono">Здесь 6
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-1
 >
 {plate.name}
 {plate.amount}
 </div>
)
)}
 </div>
)}
</div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?
 {entry}
 </div>
)}
)}
 </div>
</div>

```

```

 steps.push({ i, j, pi: [...pi], highlight:
 совпадений нет.` }));
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` }));
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` }));
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource:

```



```

 }, [mode, safePattern, safeText]));

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 const timer = useRef<NodeJS.Timeout | null>(null);

 // Reset when inputs or mode change
 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

 // Timer loop for autoPlay
 useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 }
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
 }, [autoPlay, steps.length]);

 const playPause = () => setAutoPlay(!autoPlay);
 const nextStep = () => { setAutoPlay(false); setSte
 const prevStep = () => { setAutoPlay(false); setSte
 const reset = () => { setAutoPlay(false); setStepId

 const step = steps[stepIdx] || steps[0];

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs

```

```

borderColor = "border-a
bgColor = step.match ==
-900/40");
 }
}

// Z Logic
if (mode == "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget == index) {
 borderColor = "border-a
 bgColor = step.match ==
-900/40");
 }
}

return (
 <div key={index} className=
 {/* L/R markers for Z */}
 {mode == "z" && step.l <= index && index <= step.r
 <div className="absolu
 }}
 {mode == "z" && step.r <= index && index <= step.l
 <div className="absolu
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

```

```

 <div className="abs
 <span classNam
 </div>
 })

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }}}}
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled:
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled:
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={reset} className="
border-slate-800">
 <RefreshCw strokeWidth={3} size=

```

```
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
: `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}
}
```

</pre>

</div>

</div>

);

}

---

ФАЙЛ 68/85: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";
```

```
interface TooltipProps {
```

```
 /** Содержимое подсказки (можно JSX). */
```

```
 content: React.ReactNode;
```

```
 /** Элемент, на который наводят курсор / фокус. */
```

```
 children: React.ReactNode;
```

```
 /** С какой стороны показывать. По умолчанию – сверху
```

```
 side?: "top" | "bottom";
```

```
 className?: string;
```

```
}
```

ФАЙЛ 69/85: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];
```

```
const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
```

```

 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in topological order)
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
 u,
);
 }
 }
 };

 fullyProcessed.add(u);
 dfsStack.pop();
 topoList.push(u); // prepend behavior: we write in reverse order
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

// Run DFS for all unvisited

```

```

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}

```

```

 {dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === edge.u)
 const v = dagNodes.find((n) => n.id === edge.v)

 const isActive =
 (step.currentNode === edge.u || step.currentNode === edge.v) ||
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow"})`}
 className="transition-all duration-300"
 />
);
 })}
)}
}

```

```

{ /* Nodes */ }
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id
 const isVisited = step.visited.has(node.id)
 const isProcessed = step.fullyProcessed.has(node.id)

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void strokeColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 }
})}

```



```

 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}}}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 border border-slate-900 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

```

```

<div className="bg-slate-900 border border-slate-950 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded border border-slate-950 pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (

```

```
 <button
 disabled={currentStepIdx >= steps.length}
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
 </div>
</div>
);
};
```

---

ФАЙЛ 70/85: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
```

```

 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.
 Component: SegmentTreeVisualizer,
 },

```

```

 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
 },
 "bellman-ford": {
 title: "Форд-Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
 },
 floyd: {
 title: "Флойд-Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
 "mst-kruskal": { title: "Краскал и DSU", Component: () => (
 "mst-prima": { title: "Прим", Component: () => (
 "mst-boruvka": { title: "Борувка", Component: () => (
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
 "string-z-func": { title: "Z-функция", Component: () => (
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">

```

```

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0,
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4,
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2,
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3,
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1,
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2,
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3,
 = 2 (HE)
 }
 }
 }
 }
 }
 }
 }
 }
 }
 };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('');

```

```

const char = textToScan[currentIndex];
let curr = currentNode;
let logMsg = `[Символ '${char}']`;

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
} else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лист`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr}`;
 logMsg += `рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 }
 curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

```

```

 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -

 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плавает в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 H" и "S" не подходят. Ветки нет, fail[HI] = ROOT
 codeLine: 3,
 }
}

```

```

 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С',
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
],
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNodeId];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата

```



```

 { /* Управление шагами обучения */ }
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди BFS (начиная
 ar 8 (SHE → HE), где наглядно виде
 </p>
 <div className="space-y-1.5 mb-6 overflow
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white fon
 : 'bg-slate-800 text-slate-400
 }` }
 >
 {step
 {idx === trainStep && <span classNa
 </button>
 }}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>

```

```

 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ал IF!}
 </div>
 <div className={`p-1.5 rounded flex ite
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold ml
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>
 <h5 className="text-white font-bold mb-2

```

```

 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium transition-colors
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 `}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
 <div>
 <h5 className="text-white font-bold mb-3">Текущее действие в коде</h5>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i];
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie[curr].next)
 {activeSearchCodeLine === 2 && прыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center border border-blue-400 text-blue-200 font-bold`} >

```

```

<div className="flex items-center justify-between" style={{width: 1000px}}>
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
</div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">
 <g>
 <!-- Горизонтальные линии и подписи уровней -->
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
 { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
 { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
 { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14" height="24" />

```

```

 y={{fromNode.y + toNode.y} / 2 +
 fill={isTrainingExamined ? (isMatch
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode
 .y) / 2})`}>
 <rect x="-15" y="-12" width="30
strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 {isMatchBranch ? ' ' : ' '}
 </text>
 </g>
)}
 </g>
);
 });
 }}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.fromNode];

```

```

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработали
 const stepIdxForNode = TRAINING_STEPS_INFO[node.id];
 if (activeMode === 'training' && trainStep === stepIdxForNode)

```

```

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
 }
 return (

```

```

 {isCurrent && {
 <circle cx={node.x} cy={node.y} r="24" fill={isCurrent ? '#2563eb' : '#f8d7da'} stroke={isCurrent ? '#ff0000' : '#000000'} stroke-width={isCurrent ? 2 : 1} />
 }}
 {isTargetChild && {
 <circle cx={node.x} cy={node.y} r="24" fill={isTargetChild ? '#f8d7da' : '#2563eb'} stroke={isTargetChild ? '#ff0000' : '#000000'} stroke-width={isTargetChild ? 2 : 1} />
 }}

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24' : '12'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#ffffff'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : '#000000'}
 stroke-width={isCurrent || isTargetChild ? 2 : 1}
 className="transition-all duration-300"
 />

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 12 : 10}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме обхода */}
 {isTargetChild && {
 <g transform={`translate(${node.x}, ${node.y})`}
 <rect x="-35" y="-12" width="70" height="24" fill="red" opacity="0.5" />
 <text x="0" y="2" fontSize="10" fill="white" font-weight="bold">
 ИЩЕМ FAIL
 </text>
 </g>
 }}
 </div>
</pre>

```

```

 </text>
 </g>

 </svg>
 </div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{backgroundImage: 'url(/img/gradient.svg)'}}>
 <h5 className="text-white font-bold mb-3 text-sm">
 Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">Пока ничего не поймали. Запустите шаги симуляции</p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 style={{backgroundImage: 'url(/img/gradient.svg)'}}
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-500" />
 {match.word}

 Индекс {match.index}

 </div>
))}
 </div>
)}
 </div>
)}
</div>
);

```

```

const [mode, setMode] = useState<"1d" | "2d">("1d");
return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
 }`}
 >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]], []),
 []
);
};

```

/\*\* Что сейчас под курсором: префикс P[i] или элемент A[i]

```

const [hover, setHover] = useState<{ kind: "pref" | "el"; i: number}>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number}>({ l: 0, r: 0 });

```

```

const covered =

```



```

 : "bg-slate-800 text-slate-300"
 }` }
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}
}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help $
 active
 ? "bg-emerald-500 text-white ring
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={`P[${i}] = сумма a[0..${i} - 1]}
 >
 {v}
 </div>
);
)
 })}
 </div>

```

```

 { /* Запрос */
 <div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range.l
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) п
 </p>
 </div>
 </div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i
 }
 }
 return p;
 }, [n, m]);

 const [hover, setHover] = useState<{ i: number; j: num
 const [rect, setRect] = useState({ r1: 1, c1: 1, r2:

```

```

 </div>
);
 }
 }
</div>
)
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hover?.j === j;
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner === "C"
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border-slate-500";
 return (
 <div
 key={j}
 onMouseEnter={() => setHover({ i, j })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-pointer ${
 isHover ? "bg-amber-500 text-white" : cornerColor
 }`}
 >
 {v}
 {corner && !isHover && (

 ht">

```

---

ФАЙЛ 73/85: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ

---

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
 }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
];
```

```

}

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d")

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 1. Сворачивание 1D
 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
)
}

```

```

 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" >
 <div className="text-[11px] uppercase tracking-wider" >
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto" >
 {arr1D.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1;
 return (
 <div key={idx} className="flex flex-col items-center" >
 <div
 className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em)]}
 style={inCover ? "bg-indigo-500 text-white" : ""}
 >
 {v}
 </div>

 {v}

 </div>
);
 })}
 </div>
 </div>
 </div>

```

```
<div className="text-center font-bold text-
rap">
```

```
 2^2 (L=4)
```

```
</div>
```

```
<div className="text-center font-bold text-
rap">
```

```
 2^3 (L=8)
```

```
</div>
```

```
{/* Matrix */}
```

```
{Array.from({ length: N }).map((_, i) => {
```

```
 <React.Fragment key={i}>
```

```
 <div className="text-slate-500 font-mon
```

```
 {i}
```

```
 </div>
```

```
 {Array.from({ length: LOG }).map((_, j)
```

```
 const isValid = i + (1 << j) <= N;
```

```
 const isVisible =
```

```
 isValid &&
```

```
 (j === 0 ||
```

```
 computeSteps.findIndex((s) => s.i
 step));
```

```
 // Active computing state
```

```
 let isActive = false;
```

```
 let isSrc1 = false;
```

```
 let isSrc2 = false;
```

```
 if (step > 0 && step <= maxStep) {
```

```
 const currentStep = computeSteps[ste
```

```
 if (currentStep.i === i && currentS
```

```
 isActive = true;
```

```
 if (currentStep.j === j + 1 && curre
```

```
 isSrc1 = true;
```

```
 if (
```

```
 currentStep.j === j + 1 &&
```

```
 currentStep.i === i - (1 << (curre
```

```
)
```

```

 >
 {st1D[i][j]}
 </motion.div>
)}

 {/* Arrows visualization */}
 {(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-e
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc
 fill="none"
 stroke={isSrc1 ? "#10b981"
 strokeWidth="3"
 />
 </motion.svg>
)}
 </div>
);
 }}}
</React.Fragment>
)})
</div>
</div>
</div>

 {/* Formula helper text */}
 <div className="bg-slate-900 border border-slate-

```



```
const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number }>({ r: 0, c: 0 });

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
 <h3 className="text-xl font-bold text-indigo-400">Визуализация
 <p className="text-sm text-slate-400 mb-6 text-wrap">
 Для наглядности показываем только <b className="text-slate-400">

 иц!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-l-1">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); setLocked(null); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
))}
 </div>

 <div className="flex flex-col items-center gap-6 no-scrollbar">
 {[...Array(k + 1)].map((_, i) => k - i).map((step, i) => {
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
```

```

highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur
] md:scale-[1.08]';
 } else if (!isTop && isLeft
 highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amb
3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
}

return {
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {
 if(locked && locked.
 else setLocked({r, c
 }
 }}
 className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
>
 {ST2D[r][c][step][step]}
 </div>
);
 }}}
</div>
</div>
</div>

```

```

 c + pL][k-// Правый Низ
 {'}'}));
 </div>

 {activeCell ? (
 <div className="bg-slate-950 p-5 rounded-2xl mt-2 transition-all">
 <p className="text-slate-300 mb-3">
 Пример для ячейки:
 {locked && <Lock size={
 </p>
 <p className="text-indigo-300 border">

 }}[k]][k]

 </p>

 <div className="space-y-2 pt-3 pl-2">
 {(() => {
 const prevL = 1 << (k - 1);
 return (
 <div className="flex items-center">
 </div> ST[{activeCell.r}[activeCell.c][k-1][k-1]}
 </div>
 <div className="flex items-center">
 </div> ST[{activeCell.r}[activeCell.c + prevL][k-1][k-1]}
 </div>
 <div className="flex items-center">

```

```
const [r1, setR1] = useState(1);
const [c1, setC1] = useState(1);
const [r2, setR2] = useState(5);
const [c2, setC2] = useState(6);

const H = r2 - r1 + 1;
const W = c2 - c1 + 1;
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900">
 <h3 className="text-xl font-bold text-rose-500">
 <p className="text-slate-400 text-[13px] font-normal">
 еpx, r2, c2 - правый низ).</p>
 </h3>
 <div className="flex flex-col items-center">
 <div className="grid grid-cols-[repeat(8,1fr)] gap-2">
 <div className="border border-rose-500/50 border-scale-105 z-10 p-2">
 {val2D[r][c]}
 </div>
 </div>
 </div>
 </div>
 </div>
)
```

```

 </div>
 </label>
 <label className="flex items-center" >

 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
 </div>
 </div>
</div>

```

```

<div className="flex-1 bg-slate-900 p-6 rounded" >
 <h3 className="text-xl font-bold text-emerald-400" >
 <p className="text-sm font-sans text-slate-400" >
 Мы знаем, что максимальная степень дв
 {Lx}. Аналогично
 {Ly}.
 </p>
 <p className="text-sm font-sans text-slate-400" >
 Чтобы покрыть весь прямоугольник, мы

 роса. (Цветные прямоугольники слева).
 </p>

```

```

 <div className="bg-[#0a0f1e] rounded-xl p-6" >
 <div className="text-slate-300 shadow-inner overflow-x-auto" >
 <div className="absolute right-3 top-3" >
 // 1.
 int<
 1); int<
 int<
 1); int<
 int<
 // Выс: {Lx}


```

```

 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky].text;

 let badge = null;
 if (isTL) { color = "bg-rose-400 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-blue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
 transition-all duration-300 relative shrink-0 ${cx} ${cy}
 {txt}
 {badge && <div className="text-white font-bold rounded"
 r-slate-700 text-white px-1 font-bold rounded
 </div>
 }
)
)}
 </div>
 </div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white"
 >
 <div className="text-center font-bold text-white"
 wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border-2 border-slate-800


```

```
 : 'px-3 py-1 rounded text-sm font-bold'

const aside = document.querySelector('aside')
const headerMain = document.querySelector('h1')
const questionsContainer = document.getElementById('questions')
const wtfContainer = document.getElementById('wtf')

if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display = 'none'
 if (questionsContainer) questionsContainer.style.display = 'none'
 if (wtfContainer) wtfContainer.classList.remove('hidden')

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.search)
 window.scrollTo(0, 0);
} else {
 document.body.classList.remove('wtf-mode')
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden')
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]').forEach(btn => {
 btn.classList.remove('outline', 'outline-1');
 });

 if (theme === 'dark') document.getElementById('dark-button').classList.add('active')
 if (theme === 'light') document.getElementById('light-button').classList.add('active')
```

```

 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.c
 toggleDrawer();
 }
 });
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href

```



```

 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; color: white;
 overflow: hidden; flex-shrink: 0; text-shadow: 2px 2px 0 #000,
 -1px -1px 0 #000; font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute; text-shadow: 1px 1px 0 #000,
 -1px -1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
 }
 .formula-scroll::-webkit-scrollbar {
 display: none;
 }

 mjx-container {
 max-width: 100% !important;
 }

```

```

 .card-blue { background-color: #004dcf; } .card
 .card-green { background-color: #339e35; } .card
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
 background-color: #fcf8eb !important; /* li
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Se

```

```

</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Режим
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook"
 transition-colors">Notebook</button>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="yellow">
 <path d="M0,0 L6,3 L0,6 Z" fill="yellow">
 </marker>
 <marker id="arrow-orange" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="orange">
 <path d="M0,0 L6,3 L0,6 Z" fill="orange">
 </marker>
 <marker id="arrow-pink" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="pink">
 <path d="M0,0 L6,3 L0,6 Z" fill="pink">
 </marker>
 </defs>
 </svg>

```

```

 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="10"
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin="
 </svg>
 </button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0
 background-color: #000; opacity: 0.5; z-index: 1000;

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-wrap
 justify-between align-items-center" style="background-color: #fff; padding: 10px;">

 <!-- Левое меню (Оглавление) -->
 <aside id="mobile-drawer" class="fixed inset-y-0 left-0
 translate-x-full transition-transform duration-300
 w-full lg:block lg:w-72 lg:shrink-0 lg:z-auto
 <div class="h-full flex flex-col p-6 overflow-y-auto"

```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">

 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

```

```
<div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
 <a href="#q8-5" class="bloc
 <a href="#q8-6" class="bloc
 <a href="#q8-7" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-9" class:
er-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q9-1" class="bloc
 <a href="#q9-2" class="bloc
 <a href="#q9-3" class="bloc
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-400">
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2 text-fuchsia-500">
```

```

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blur

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

 ▲ Ликбез: Алгоритмы выживания

 </nav>
 <script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail

```



```

 const corresponding
 if (correspondingLi
 const details =
 if (details &&
 details.open
 document.qu
 if (othe
 othe
 }
 }));
 }
}
});
}, {
 root: null,
 rootMargin: '-10% 0px -70%
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw,
 const modal = document.createEl
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = '
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'co
 modal.style.alignItems = 'cente
 modal.style.justifyContent = 'c

```

```

copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 1000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

```

```

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);
}

```



```

 } else if (theme === 'terminal') {
 body.classList.add('bg-black');
 document.getElementById('theme-selector').innerHTML = `
 .bg-black .bg-slate-950 {
 background-color: transparent;
 }
 body.bg-black section header,
 body.bg-black button {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 body.bg-black .text-slate-950 {
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style);
 }
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1px border-slate-950 p-4">
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-950 font-medium">
 <button onclick="downloadWord()"
 font-bold flex items-center justify-center gap-2">
 Word
 </button>
 <a href="algebra.pdf" target="_blank"
 -sm font-bold flex items-center justify-center gap-2">

```

## ВЫСШАЯ АЛГЕБРА

</h1>

<p class="text-slate-400 italic text-lg

<!-- Print Table of Contents -->

<div class="hidden print:block mt-8 text-  
fter: always;">

<h2 class="font-bold text-xl mb-4">

<ul class="text-base space-y-2">

<li><a href="#question-1">1. Ал

<li><a href="#question-2">2. Ко

<li><a href="#question-3">3. Тр

<li><a href="#question-4">4. Ст

<li><a href="#question-5">5. Гр

<li><a href="#question-6">6. Де

<li><a href="#question-7">7. Ал

<li><a href="#question-8">8. Кл

<li><a href="#question-9">9. Ко

<li><a href="#question-10">10. ,

<li><a href="#question-11">11. <

<li><a href="#question-12">12. l

<li><a href="#question-13">13. l

<li><a href="#question-14">14. l

<li><a href="#question-15">15. <

<li><a href="#question-17">17. l

<li><a href="#question-18">18. >

<li><a href="#question-19">19. l

<li><a href="#question-20">20. l

<li><a href="#question-21">21. l

<li><a href="#question-22">22. l

</ul>

</div>

</header>

<!-- ===== WTF CONTAINER =====>

<div id="wtf-container" class="hidden">

<h2 class="text-3xl font-black text-cen

```

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структур
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->

```

```


 <a href="#question-18" onclick="setf
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-50
 <div class="font-bold text-slate-700"

 <a href="#question-19" onclick="setf
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emerald-500
 <div class="font-bold text-slate-700"

 <a href="#question-20" onclick="setf
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-500
 <div class="font-bold text-slate-700"

 <a href="#question-21" onclick="setf
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo-500
 <div class="font-bold text-slate-700"

 <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500
 <div class="font-bold text-slate-700"

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

 <a href="#proof-algorithms" onclick="setf
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500
 <div class="font-bold text-slate-700"

 </div>

```

```

<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-fuchsia" style="color: #f080f0;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

i>

 т.д.).

что так и задумано: "Ой, противоречие, теорема".

 </div>

```

```

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-emerald" style="color: #2ecc71;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

, r_2).

счет ограничений (размер остатка) докажи, что не существует.

 </div>

```

```

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-yellow" style="color: #f1c40f;">
 <p class="text-sm text-slate-400" style="color: #a0b0c0;">
 <ul class="list-disc pl-5" style="list-style-type: none;">

валось.

Просто перепиши формулу с буквой k .

усок k , замени его на жареного карася из пункта 1.


```



```
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
ициентов (они делятся на p), кроме старшего,
зложить.</p>
```

```
</div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400 font-medium">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium text-slate-400">
. Чаще всего после этого Фейс-контроль p .
```

```
</div>
```

```
</div>
```

```
 <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400 font-medium">
 <p class="text-sm text-slate-400">
 <code class="text-x"
 </div>

```

```
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400 font-medium">
 <p class="text-sm text-slate-400">
х двух, потом последних.

 <code class="text-x
```

Универсальное пособие • полный исходный код

---

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 77/85: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся —
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText
```

```

}
`;

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string): { body: string; termIds: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем текст как есть */
 }

 const ids: string[] = [];
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разбросан, и не работает
 host.querySelectorAll("details").forEach((d) => d.setOpen());

 return { body: host.innerHTML, termIds: ids };
}

```

```

<div style="font-size:.7rem;text-transform:uppercase"
<h1 style="color:#fff;font-size:1.6rem;font-weight:
<div style="font-size:.72rem;color:#475569;margin-t
</header>

<div class="export-note">
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(op
</div>

<article class="prose prose-invert max-w-none">
${opts.body}
</article>

${opts.gloss}

<footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?.[1];

```

```

const body = `
```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

-----  
ФАЙЛ 78/85: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```

/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
```

```

});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && !
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
 ge)} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi)
 .replace(/<[>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length,);
console.log("Терминов в глоссарии:", GLOSSARY.length,);
console.log("Не встретились:", GLOSSARY.filter((g) => !ids.has(g)).length,);

```

```
 CreationDate: new Date(),
 },
));

 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);

 for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
 }

 doc.end();
 await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
 });

 console.log("[3/3] png -> pdf");
 console.log(`      страниц: ${pages.length}`);
 console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
 }

 main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
 });
```

---

ФАЙЛ 81/85: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.2 КБ

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
```



```

 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileType: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))

```

```
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { timeZoned: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОММЕНТАРИИ)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalBytes)}`);
push(`Глав/билетов в пособии:         ${chapters.length}`);
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")}. ${rel}`);
});
push();
```

```
main();
```

ФАЙЛ 82/85: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают на диск) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=0 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
}
```

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT,
 PAGES_DIR,
 TXT_PATH,
 PAGE,
 ensureDir,
 humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/<\/p>
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/<\/p>
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? " " : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}
```

```

 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`      отрендерено ${done}\n`);
 }
 }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(", ")}`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE.pngFile)}`);

```

```
pages: write
id-token: write
```

```
concurrency:
 group: pages
 cancel-in-progress: true
```

```
jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist
```

```
deploy:
```

## Универсальное пособие • полный исходный код

```

#
└─ full_code_all_page
#
Готовые TXT и PDF коммитятся обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умол
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}`
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[
 steps:
 - name: Checkout
 uses: actions/checkout@v6
```

- ```
run: npx tsc --noEmit
continue-on-error: true
```
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | `\${ls tmp/export-pages}`"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30
 - name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7
 - name: Commit rebuilt export back to the branch
run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.*